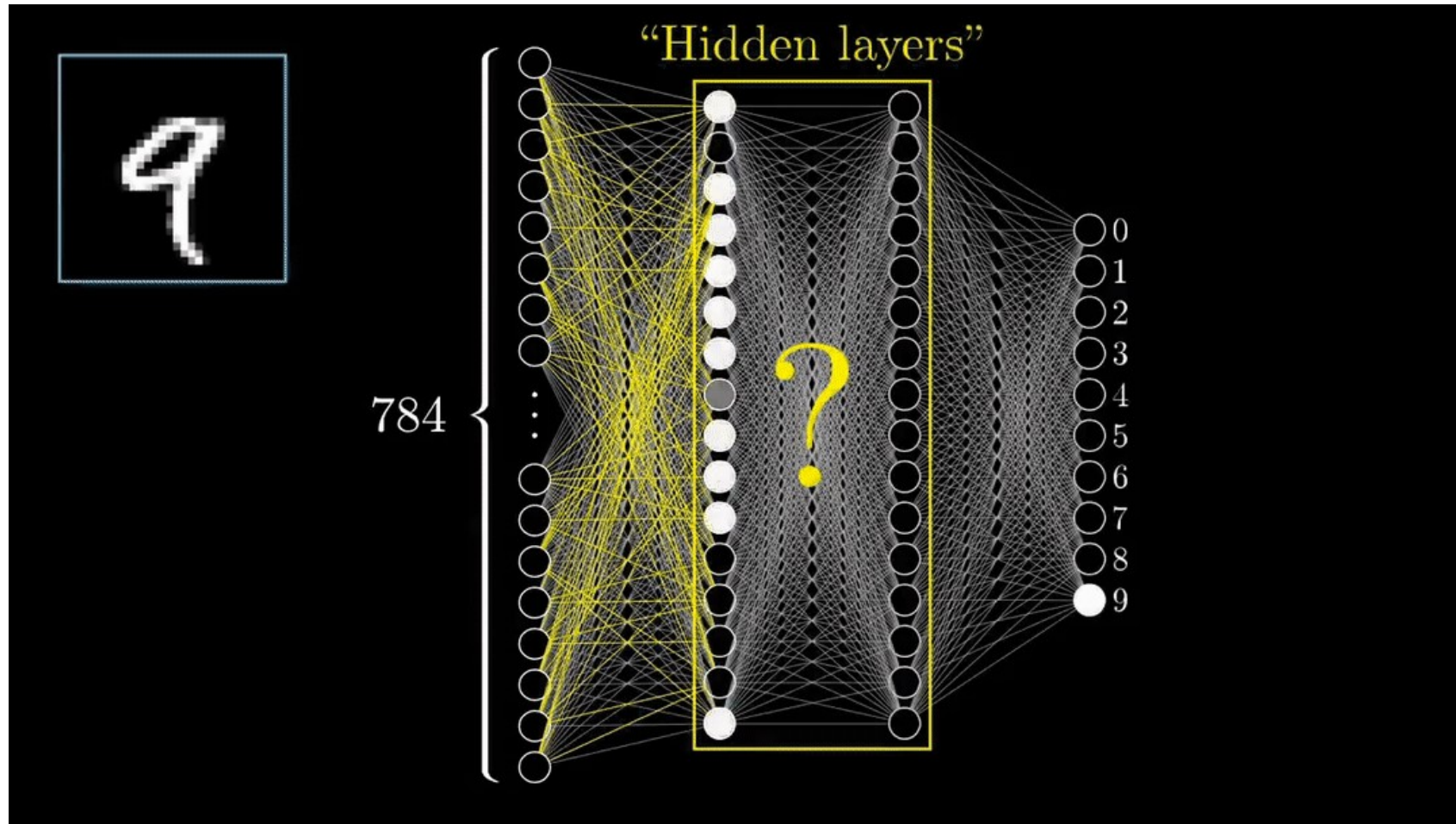


Intro to AI for Medicine

Lec 2: Math

1980s/1990s: Backpropagation



Where we left off...

Today's Agenda

1. Tips for success
2. Scalars, vectors
3. Matrices and matrix operations
4. Tie it back in with the neuron
5. Gradient descent

Housekeeping

- Attendance tracker in the drive – if zoom picked you up I added you
- Matt will take attendance this lecture
- If you're taking for credit and not on there Imk

Housekeeping: Python Gang

Free Course

Learn Python 2

Learn the basics of the world's fastest growing and most popular programming language used by software engineers, analysts, data scientists, and machine learning engineers alike.

4.3 ★★★★★ 7,397 ratings

Start

 1,098,369 learners enrolled

1 Python Syntax

In this course, learn about the syntax of the Python programming language!

-  Lesson Python Syntax
-  Project Tip Calculator
-  Quiz Python Syntax
-  Article Make the Most of Your Codecademy Membership

2 Strings and Console Output

This course will introduce you to strings and console output in Python.

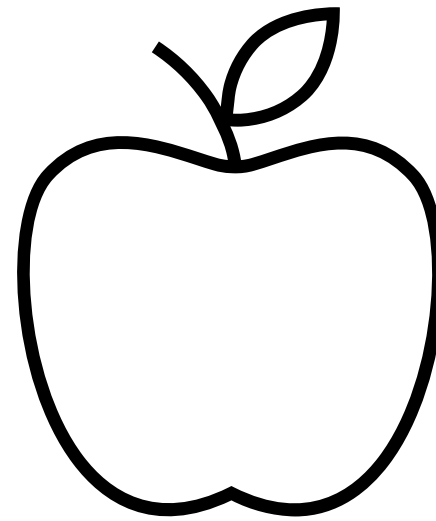
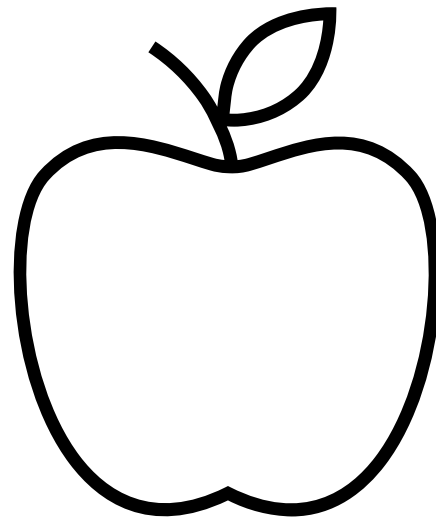
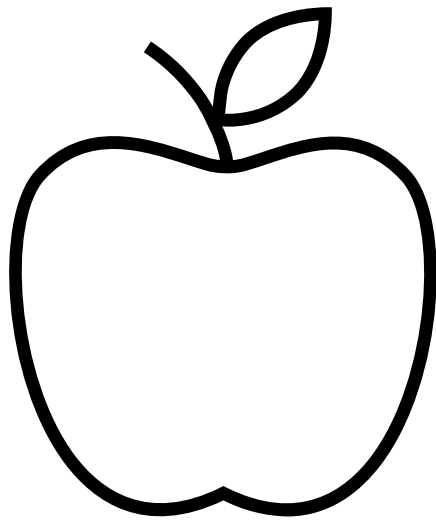
-  Lesson Strings & Console Output
-  Lesson Date and Time
-  Project Python Mad Libs
-  Quiz Strings and Console Output

OH with Matt for python after class,
OH with me for AI after class.

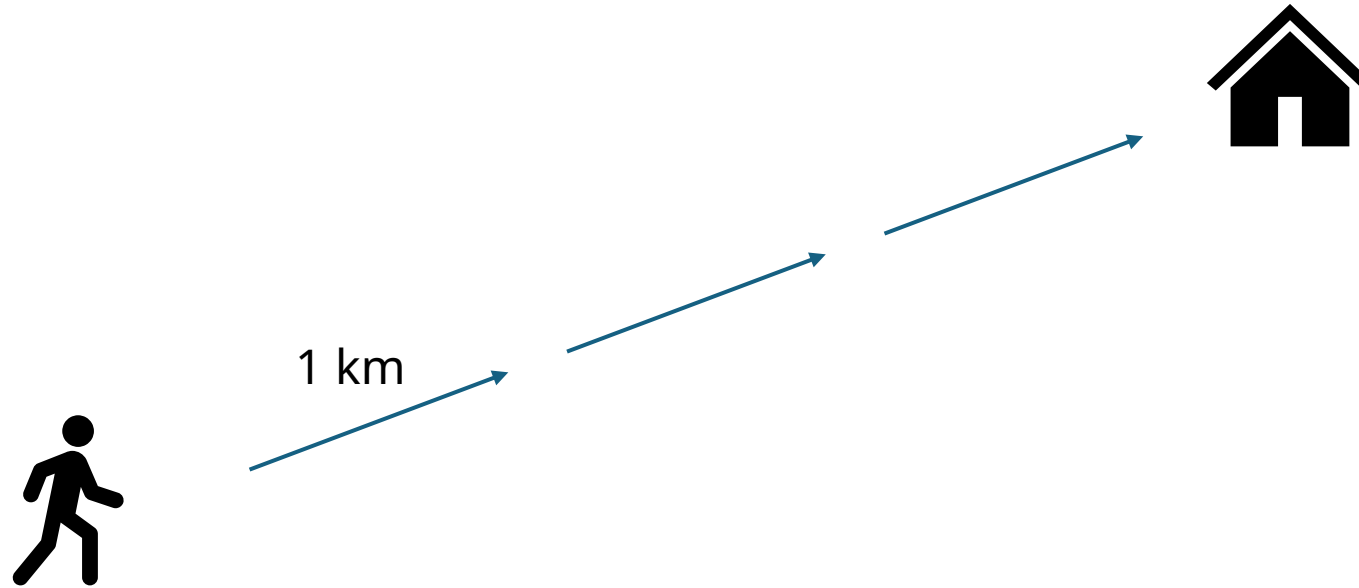
Tips

- Let's take it slow
- Try to understand how the pieces connect
- Long-term, not necessary to remember everything about this stuff
 - We're just trying to understand where the basic building blocks of ML come from
- General theme: Math has meaning
- Ask questions!

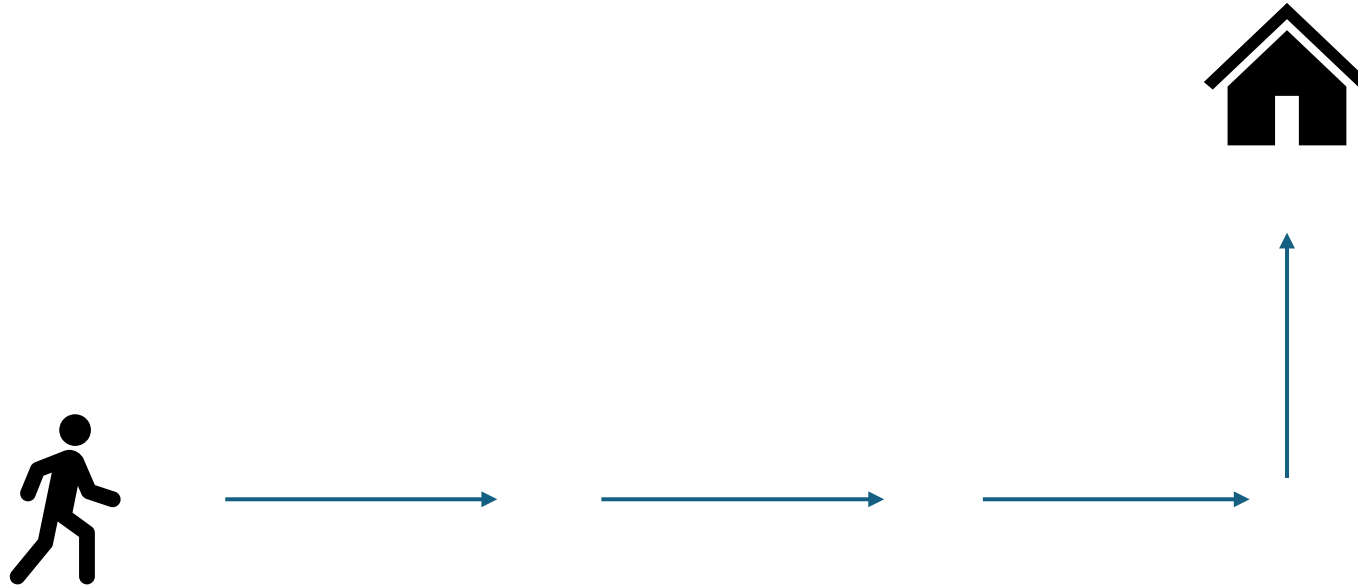
Scalar vs Vector



Scalar vs Vector



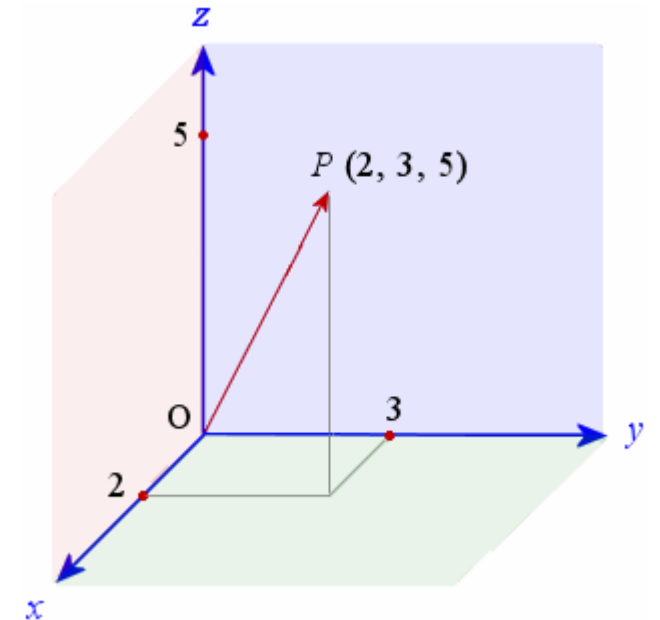
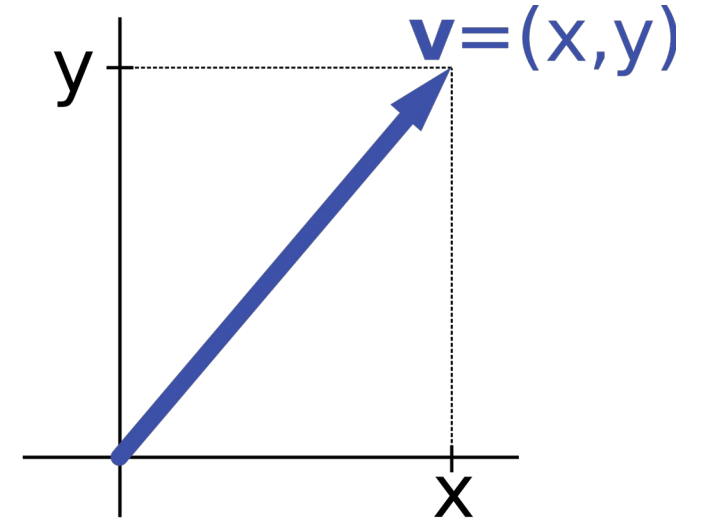
Scalar vs Vector



Vectors

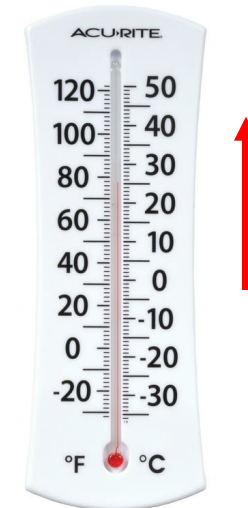
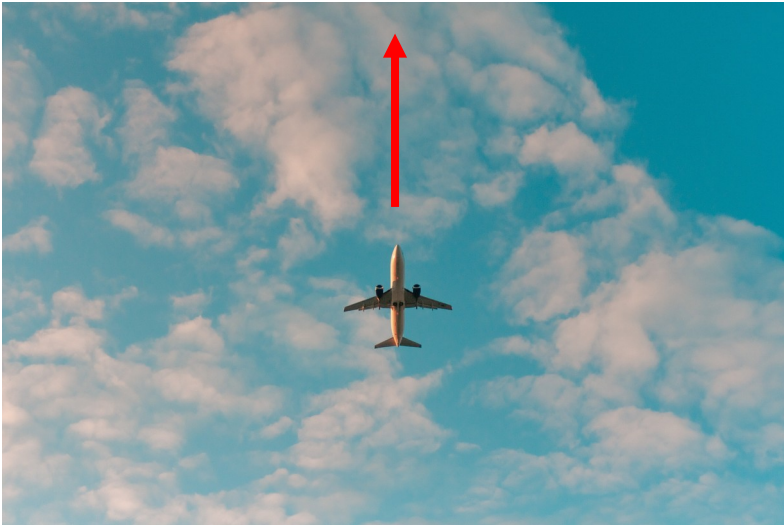
- Mathematical object with direction + magnitude
 - Direction: like slope
 - Magnitude: length of hypotenuse of triangle w legs x , y
- If no direction, it's called a scalar.
- Can have many dimensions

$$y = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}$$

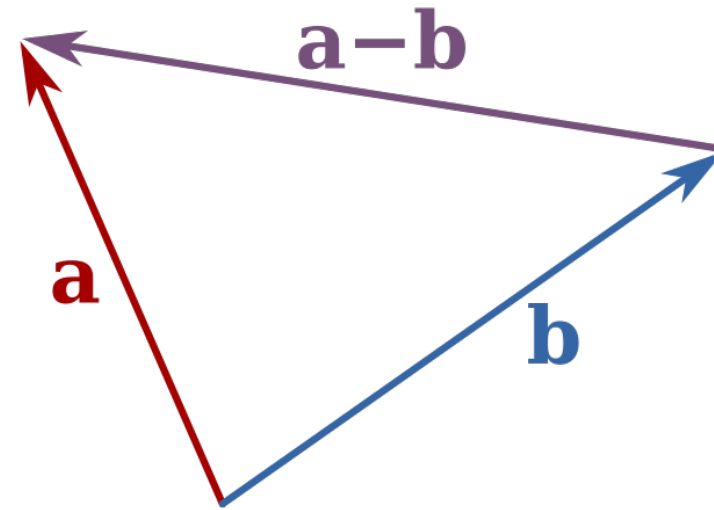
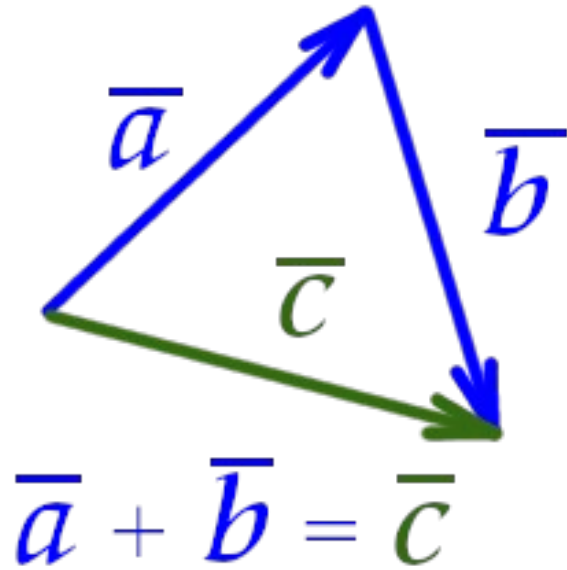


Vector Representations

- Anything with direction and magnitude!
 - Movement, force, velocity, acceleration



Vector Operations: Addition and Subtraction



Never used in AI – we're blasting past these

Vector Operations: Computing Magnitude

$$\text{Magnitude} = \sqrt{x^2 + y^2} \quad (\text{for 2D vectors})$$

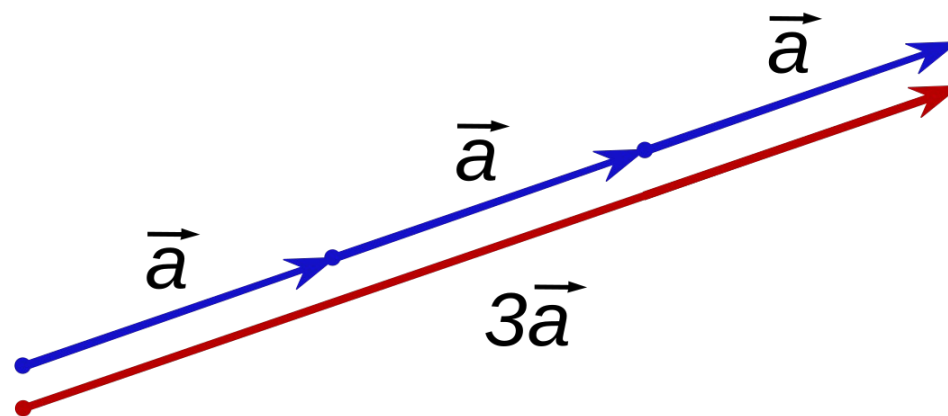
For vector $\langle 2, 2 \rangle$, $\sqrt{8}$

$$\text{Magnitude} = \sqrt{x^2 + y^2 + z^2} \quad (\text{for 3D vectors})$$

Vector Operations: Scalar Product

$$\vec{a} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$$3 * \vec{a} = 3 * \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 3*2 \\ 3*1 \end{bmatrix} = \begin{bmatrix} 6 \\ 3 \end{bmatrix}$$



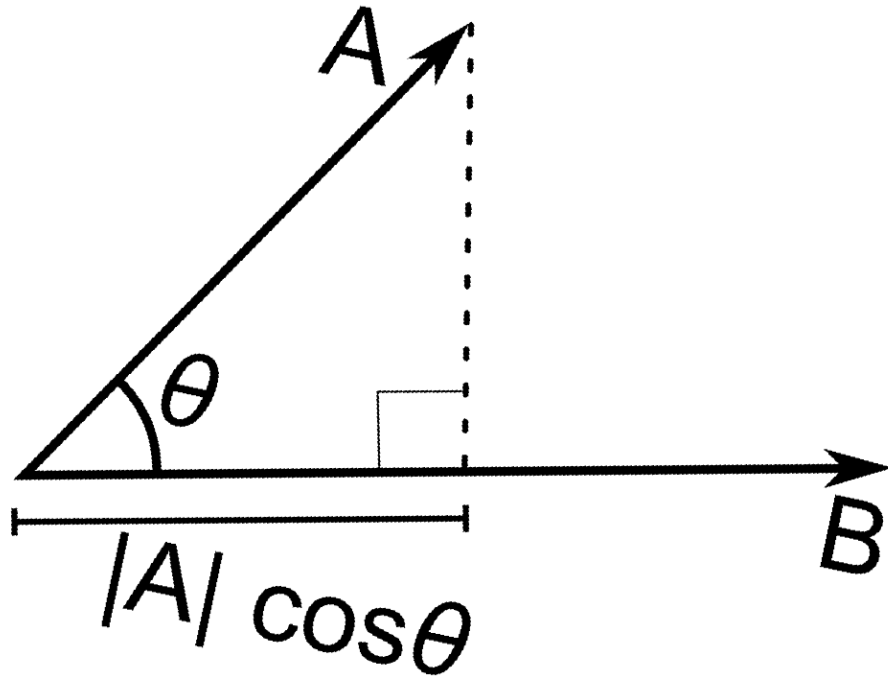
Vector Operations: Dot Product

$$\begin{bmatrix} 2 \\ 7 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} 8 \\ 2 \\ 8 \end{bmatrix} = 2 \cdot 8 + 7 \cdot 2 + 1 \cdot 8 = 38$$

Dot product

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n$$

Vector Operations: Dot Product



$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n$$

$$\begin{bmatrix} 2 \\ 2 \end{bmatrix} \cdot \begin{bmatrix} 4 \\ 0 \end{bmatrix} = 4 \cdot 2 + 2 \cdot 0 = 8$$

$$\sqrt{8} \cdot 4 \cdot \cos(45) = 2\sqrt{2} \cdot 4 \cdot \frac{\sqrt{2}}{2} = 4 \cdot (\sqrt{2})^2 = 8$$

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$$

Matrices

- Extension of vector to 3d

Bundle of vectors!

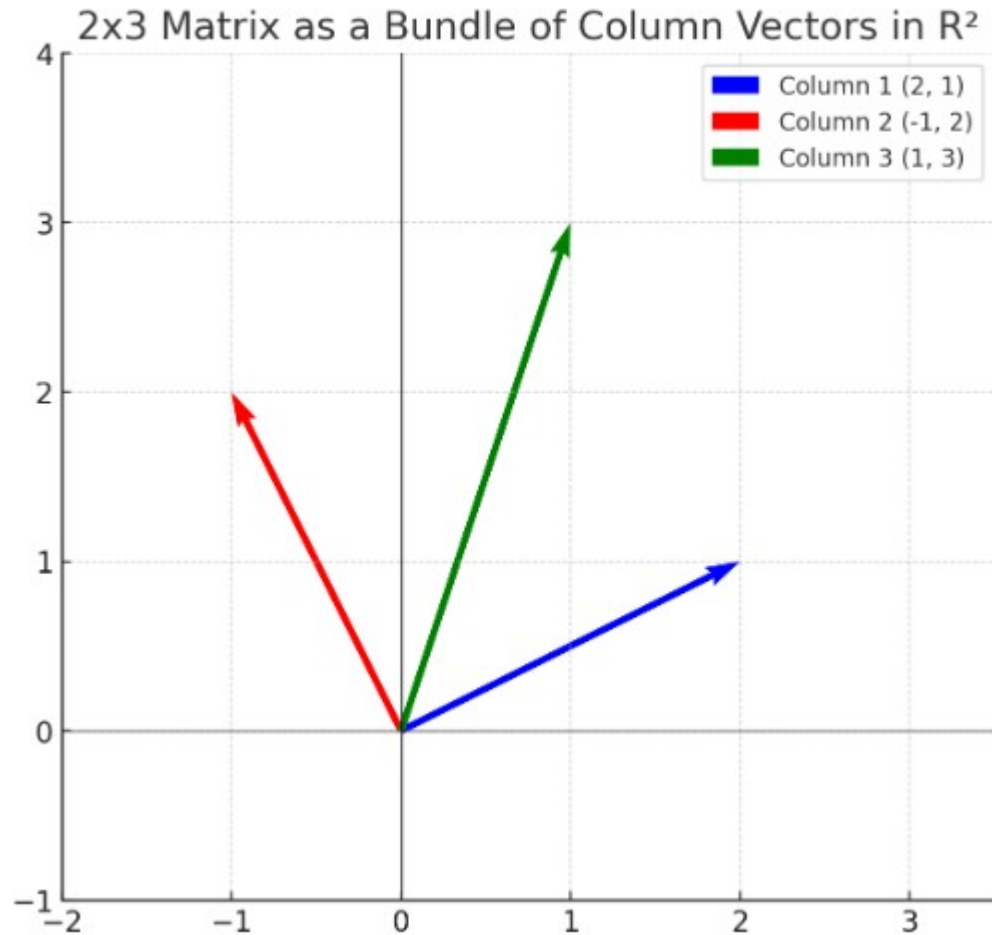
$$A = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix} = \left[\underbrace{\begin{bmatrix} 1 \\ 2 \end{bmatrix}}_{\text{col 1}}, \underbrace{\begin{bmatrix} 3 \\ 4 \end{bmatrix}}_{\text{col 2}}, \underbrace{\begin{bmatrix} 5 \\ 6 \end{bmatrix}}_{\text{col 3}} \right]$$

$$\begin{matrix} & \textcolor{red}{1} & \textcolor{red}{2} & \dots & \textcolor{red}{n} \\ \textcolor{green}{1} & a_{\textcolor{green}{1}\textcolor{red}{1}} & a_{\textcolor{green}{1}\textcolor{red}{2}} & \dots & a_{\textcolor{green}{1}\textcolor{red}{n}} \\ \textcolor{green}{2} & a_{\textcolor{green}{2}\textcolor{red}{1}} & a_{\textcolor{green}{2}\textcolor{red}{2}} & \dots & a_{\textcolor{green}{2}\textcolor{red}{n}} \\ \textcolor{green}{3} & a_{\textcolor{green}{3}\textcolor{red}{1}} & a_{\textcolor{green}{3}\textcolor{red}{2}} & \dots & a_{\textcolor{green}{3}\textcolor{red}{n}} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ \textcolor{green}{m} & a_{\textcolor{green}{m}\textcolor{red}{1}} & a_{\textcolor{green}{m}\textcolor{red}{2}} & \dots & a_{\textcolor{green}{m}\textcolor{red}{n}} \end{matrix} \begin{bmatrix} \\ \\ \\ \\ \\ \end{bmatrix}$$

The n by m matrix
A

Row matrix	$1 \times n$	$\begin{bmatrix} 3 & 7 & 2 \end{bmatrix}$
Column matrix	$n \times 1$	$\begin{bmatrix} 4 \\ 1 \\ 8 \end{bmatrix}$

Matrices: Visualized



$$A = \begin{bmatrix} 2 & -1 & 1 \\ 1 & 2 & 3 \end{bmatrix}$$

Matrices: Notation



$$\begin{bmatrix} -2 & 7 & 11 & 0 & 3 \\ 19 & 22 & -8 & 5 & 21 \\ 4 & 16 & 24 & -12 & 1 \\ 31 & 27 & 9 & 43 & -18 \end{bmatrix}$$



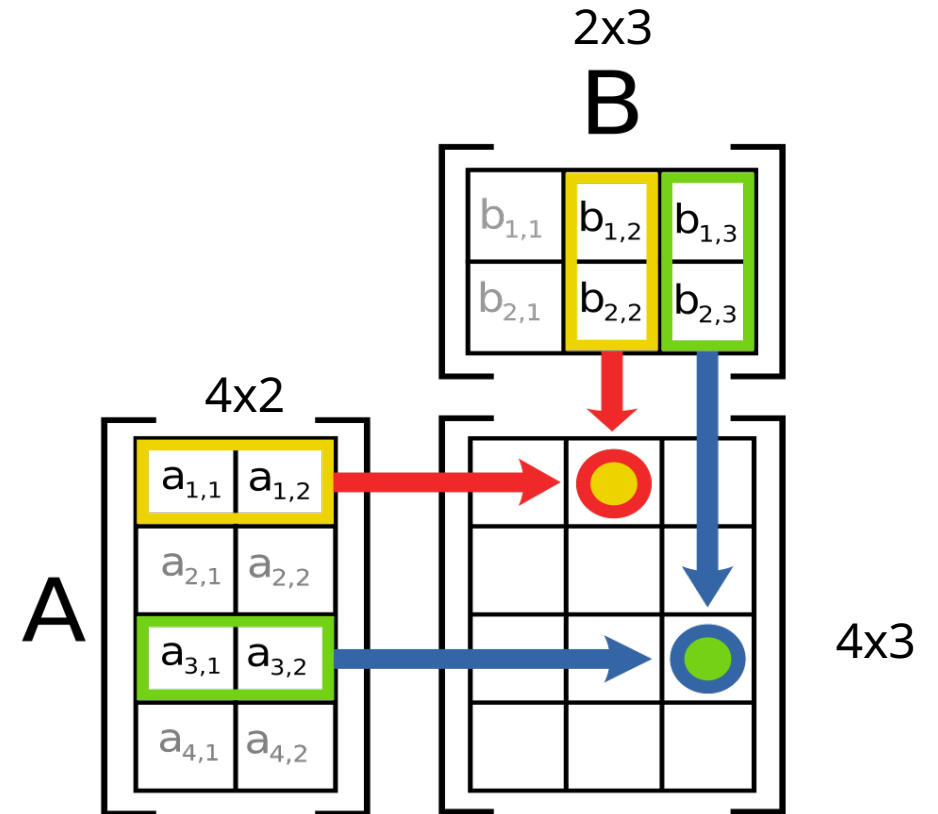
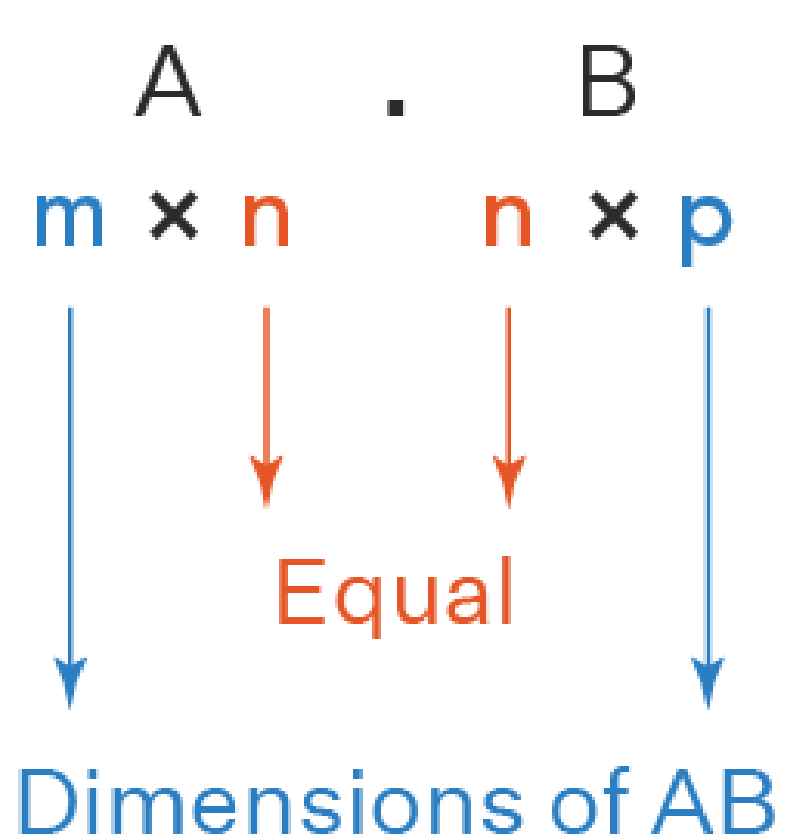
Matrices: Multiplication

$$\begin{matrix} & A & & B & & C \\ \begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} & \begin{bmatrix} b_1 \\ b_4 \\ b_7 \end{bmatrix} & \begin{bmatrix} b_2 & b_3 \\ b_5 & b_6 \\ b_8 & b_9 \end{bmatrix} & = & \begin{bmatrix} c_1 & c_2 & c_3 \\ c_4 & c_5 & c_6 \\ c_7 & c_8 & c_9 \end{bmatrix} \end{matrix}$$

$$a_1 * b_1 + a_2 * b_4 + a_3 * b_7 = c_1$$

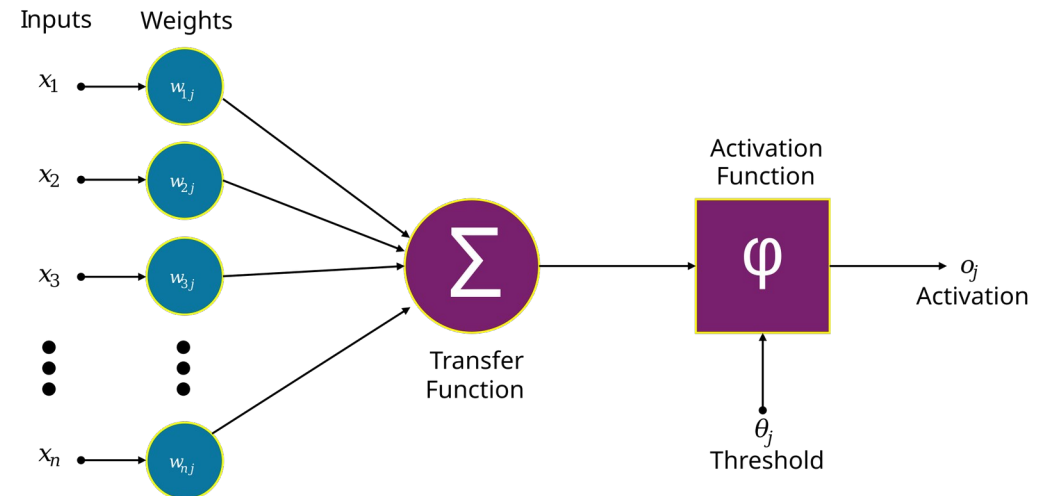
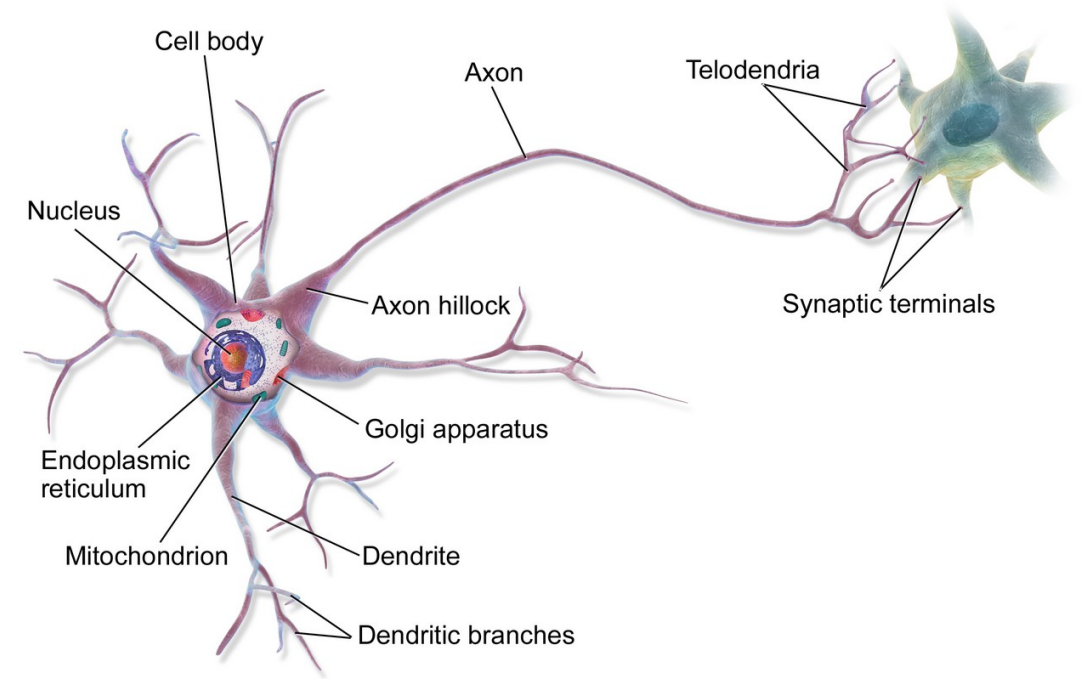
$$[\mathbf{AB}]_{i,j} = a_{i,1}b_{1,j} + a_{i,2}b_{2,j} + \cdots + a_{i,n}b_{n,j} = \sum_{r=1}^n a_{i,r}b_{r,j},$$

Matrices: Multiplication Rules

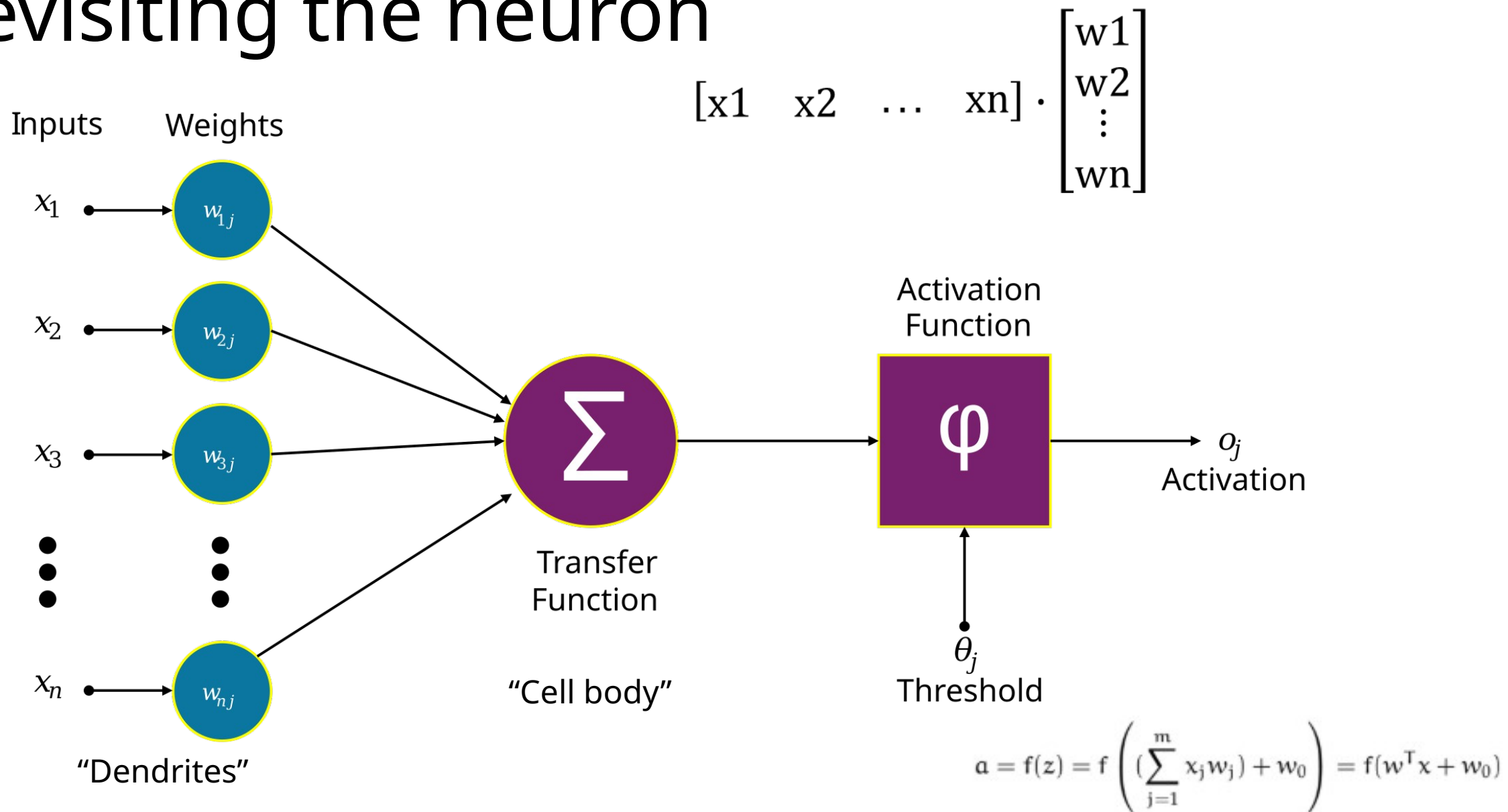


Medicine Inspires AI

- 1940s – The neuron cell is discovered
- Computer scientists wonder – can this be replicated to form the basis of Artificial intelligence?



Revisiting the neuron



Matrices: Multiplication

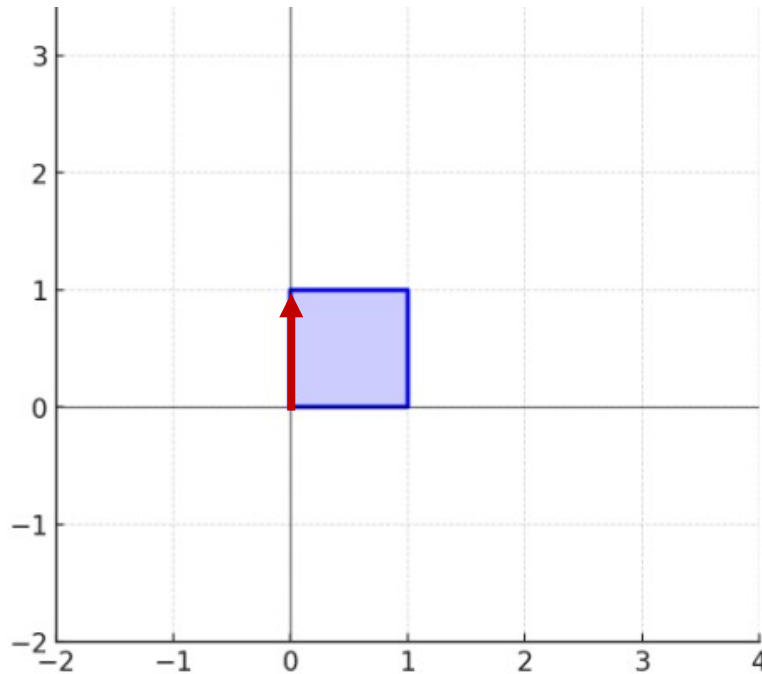
Points:

(0,0)

(1,0)

(1,1)

(0,1)



Transform Matrix

$$S = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

Outputs:

(0,0)

(2,0)

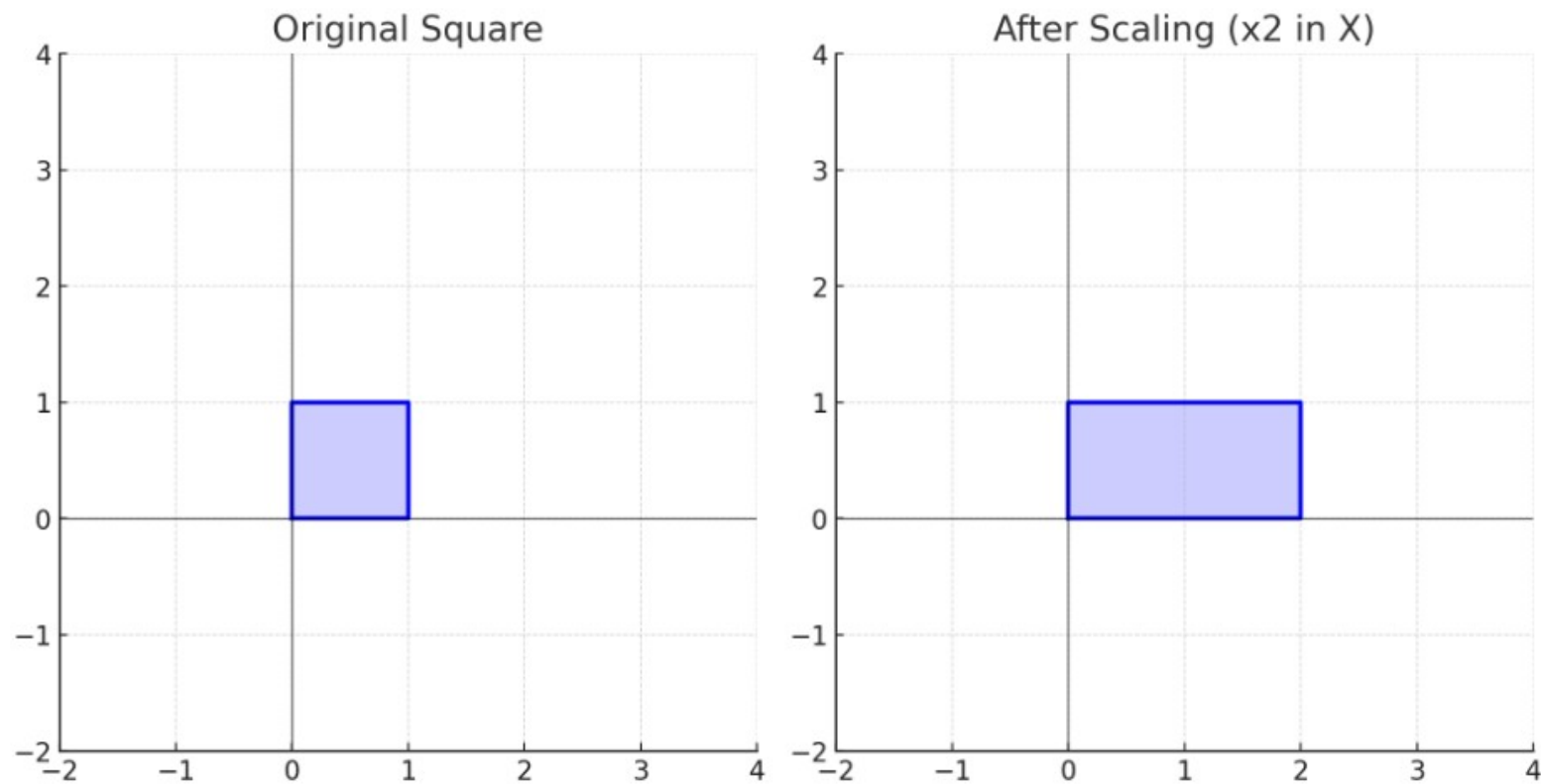
(2,1)

(0,1)

Matrices: Multiplication

Points:

(0,0)
(1,0)
(1,1)
(0,1)



Outputs:

(0,0)
(2,0)
(2,1)
(0,1)

Matrices: Multiplication

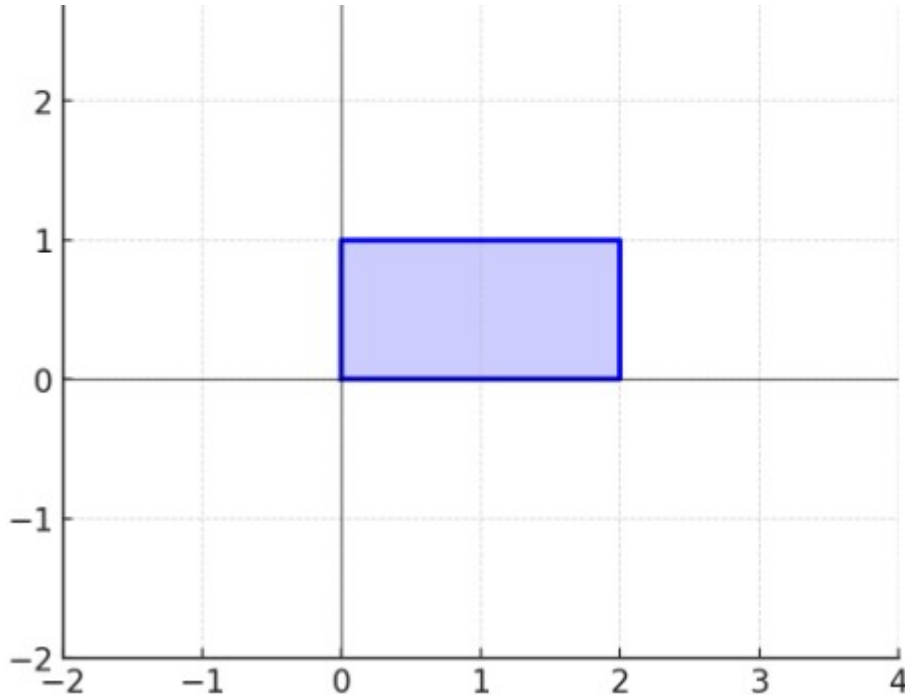
Points:

(0,0)

(2,0)

(2,1)

(0,1)



Transform Matrix

$$H = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

Outputs:

(0,0)

(2,0)

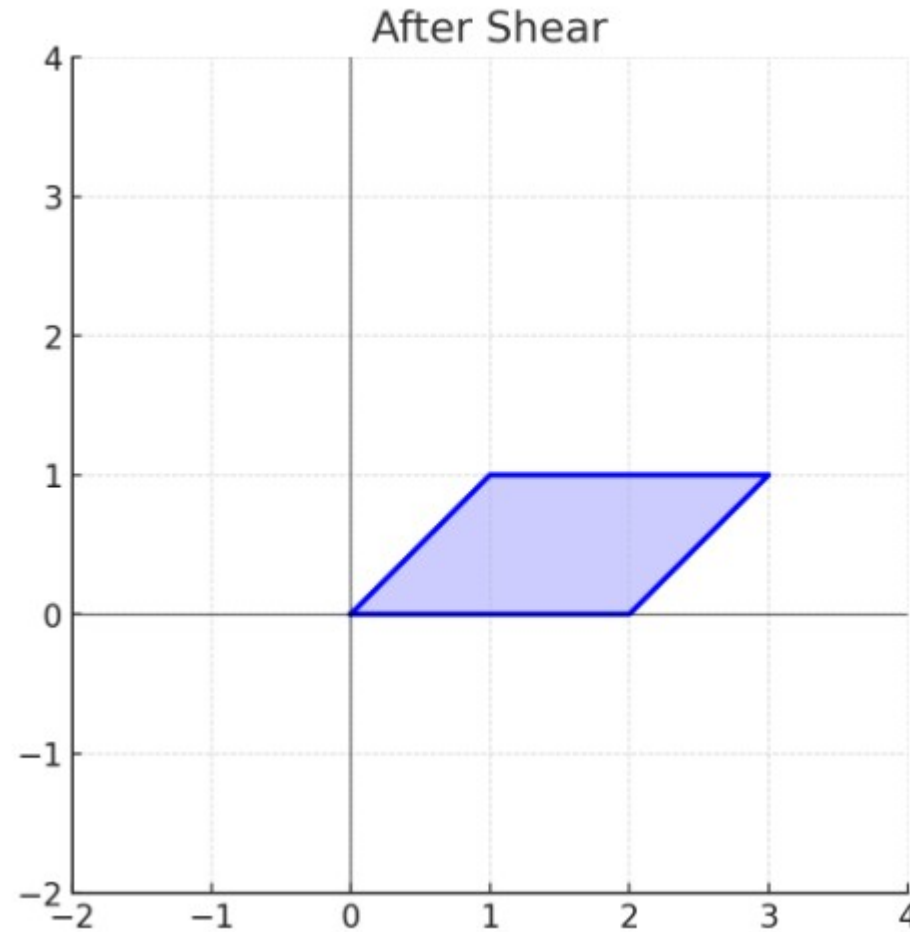
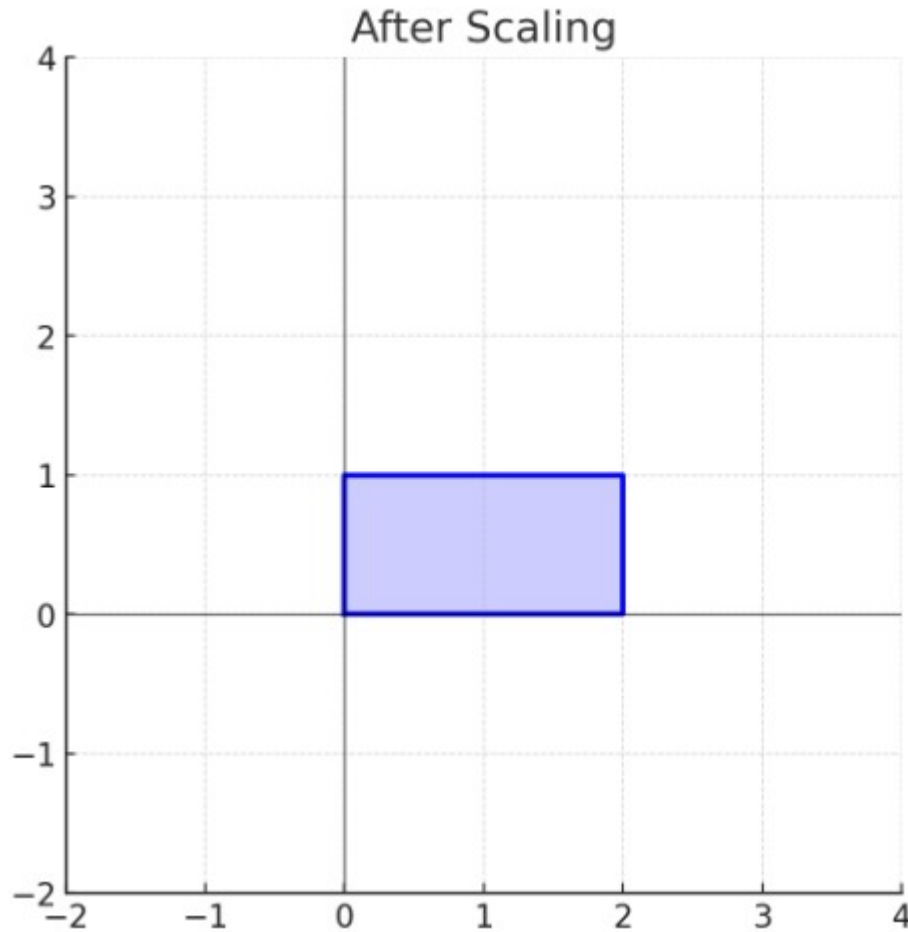
(3,1)

(1,1)

Matrices: Multiplication

Points:

(0,0)
(2,0)
(2,1)
(0,1)



Outputs:

(0,0)
(2,0)
(3,1)
(1,1)

Matrices: Multiplication

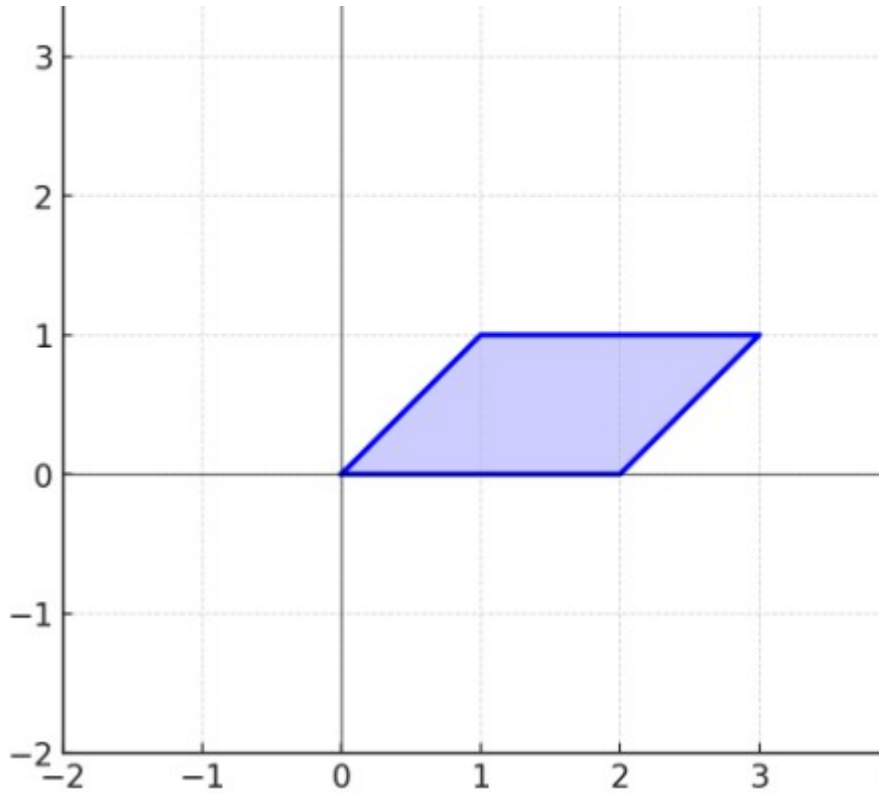
Points:

(0,0)

(2,0)

(3,1)

(1,1)



Transform Matrix

$$R = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 3 \end{bmatrix}$$

Outputs:

(0,0)

(2,0)

(3,1)

(1,1)

Matrices: Multiplication

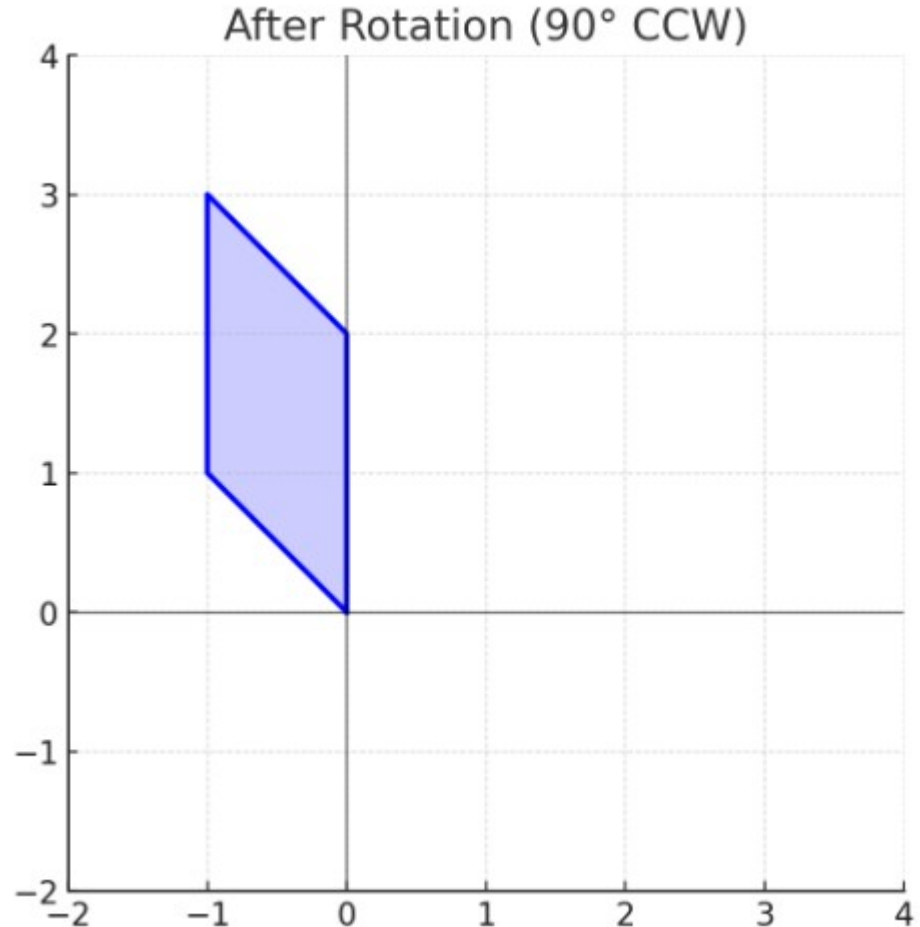
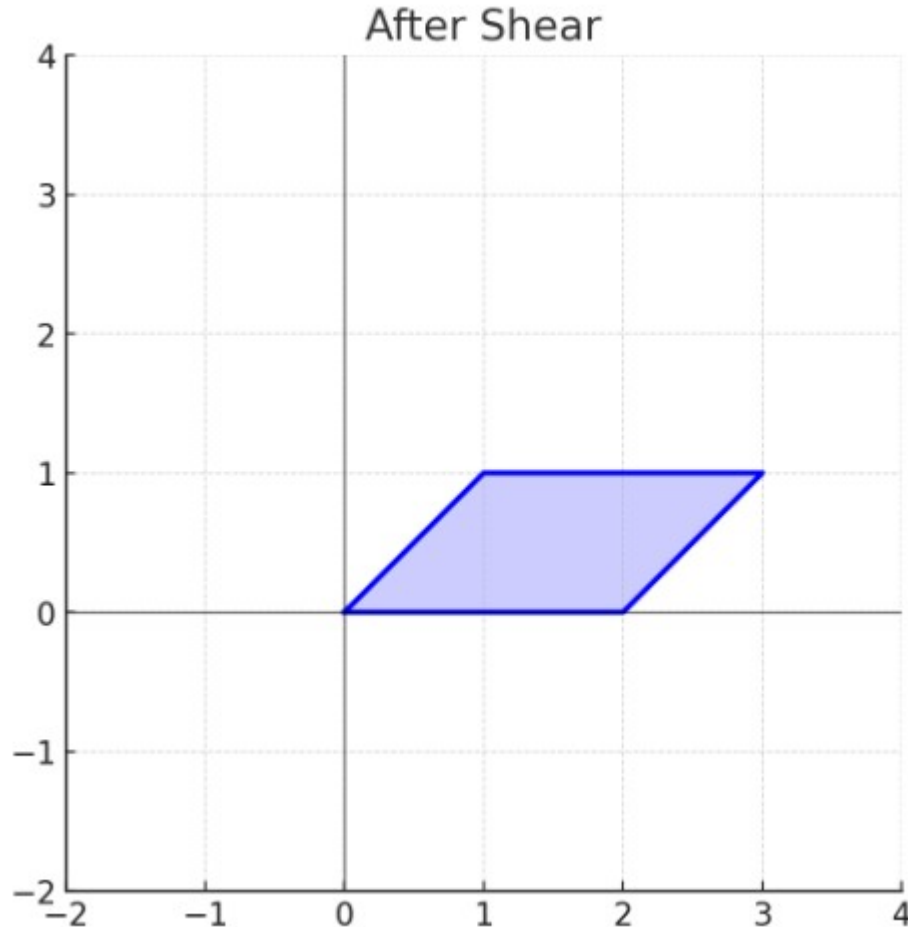
Points:

(0,0)

(2,0)

(3,1)

(1,1)



Outputs:

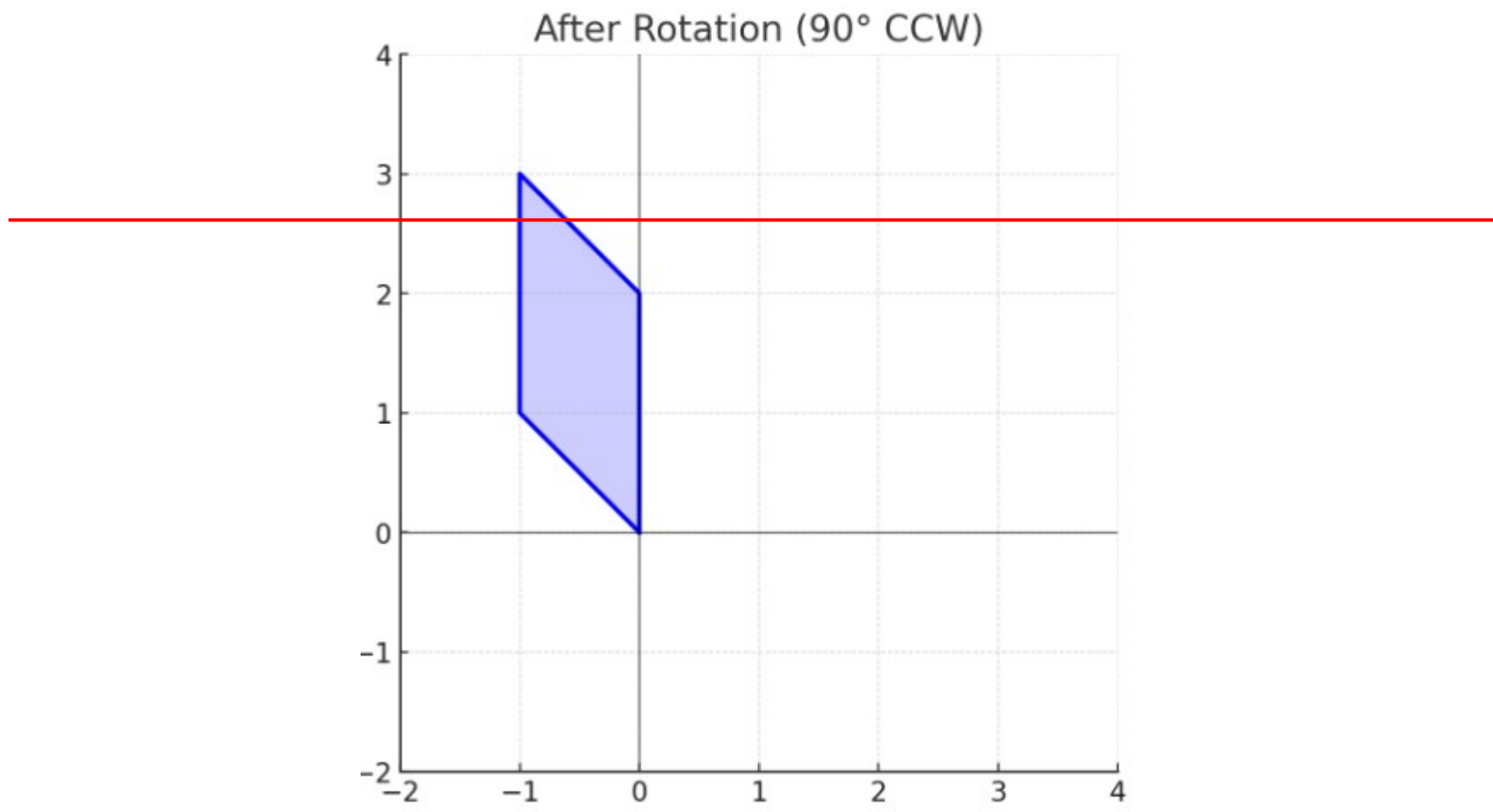
(0,0)

(2,0)

(3,1)

(1,1)

Who Cares?



What If...

Instead of
inputs:

(0,0)

(1,0)

(1,1)

(0,1)

Our inputs were:

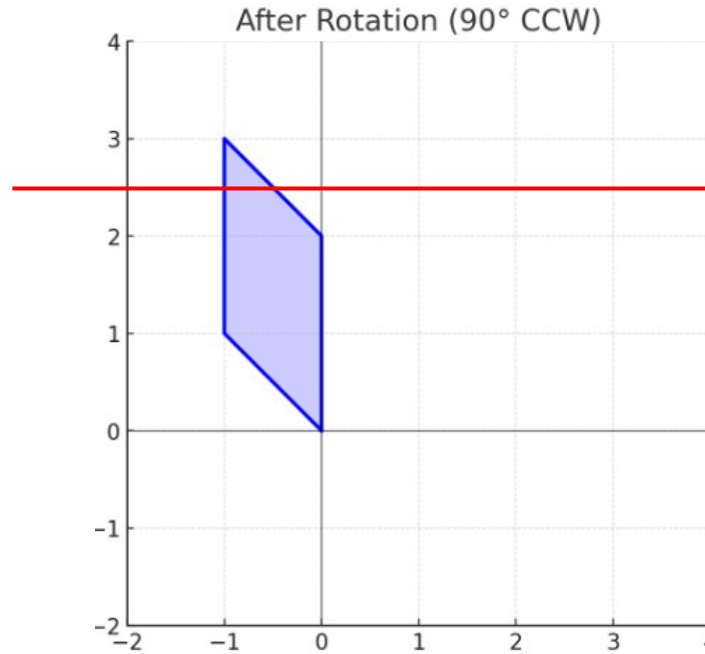
Age: 72

Weight: 133 kg

Max systolic bp last 24h: 172

Family history of atrial fibrillation:

yes



And our red line meant:
80% chance of developing
afib within 5 years

So, one point is:
(72, 133, 172, 1)

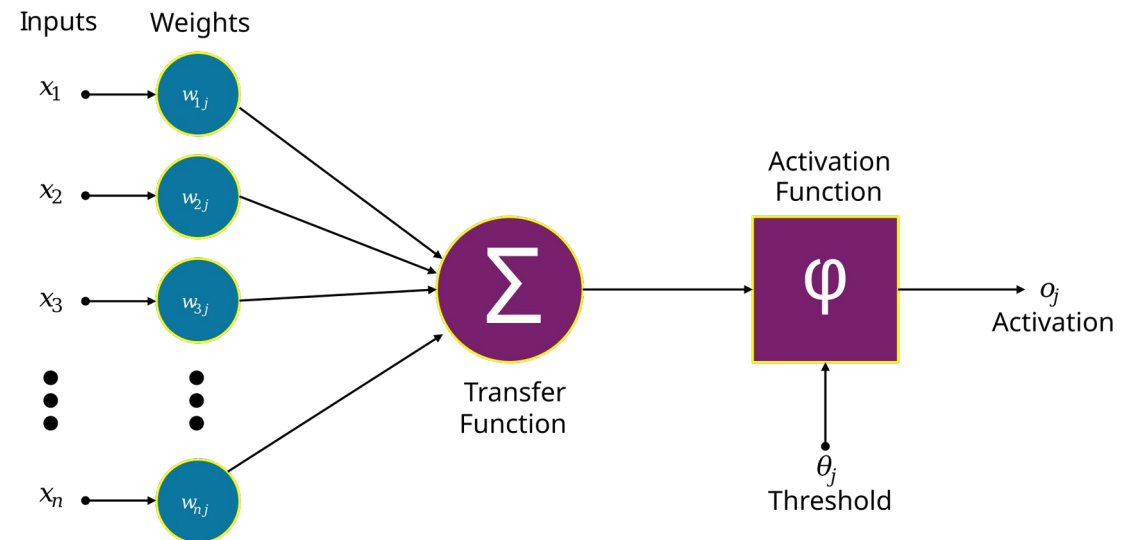
Activation

Combining our matrices from before

gives:

$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \cdot \left(\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \right) = \begin{bmatrix} 0 & -1 \\ 2 & 1 \end{bmatrix}$$

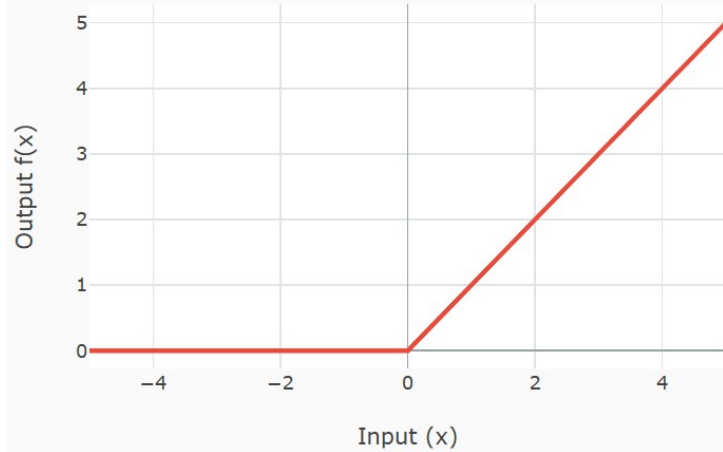
So why do multiple steps?



Activation

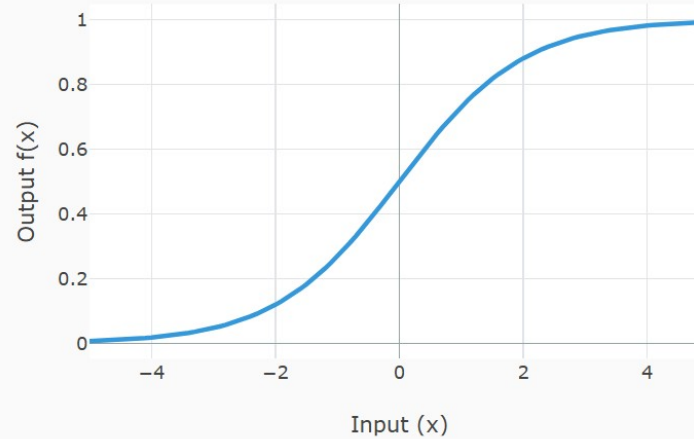
ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$



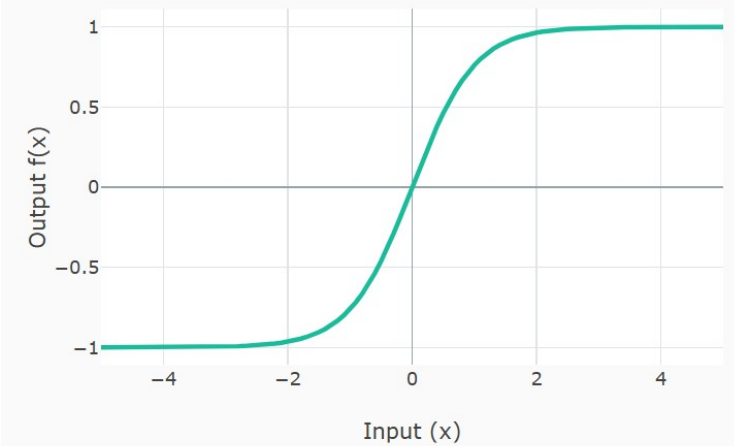
Sigmoid

$$f(x) = 1 / (1 + e^{-x})$$



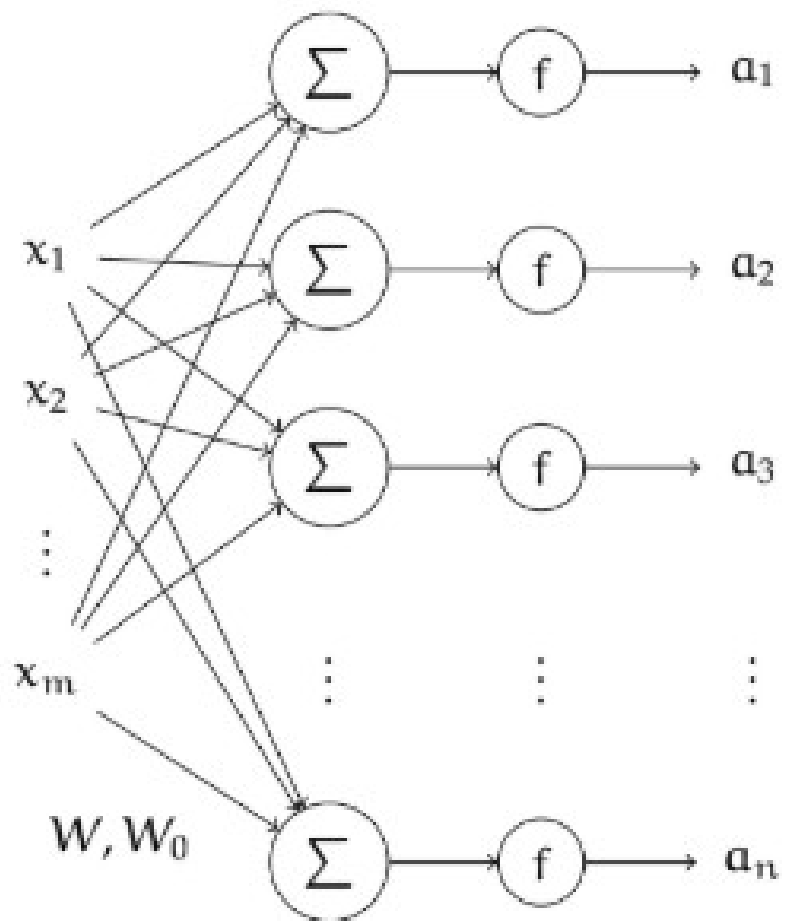
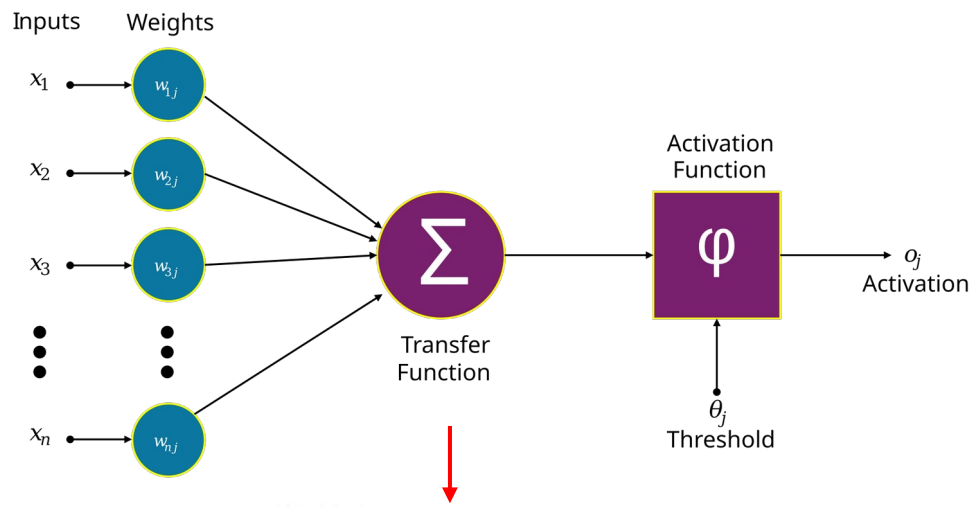
Tanh

$$f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$$



Introduce nonlinearity into the system!

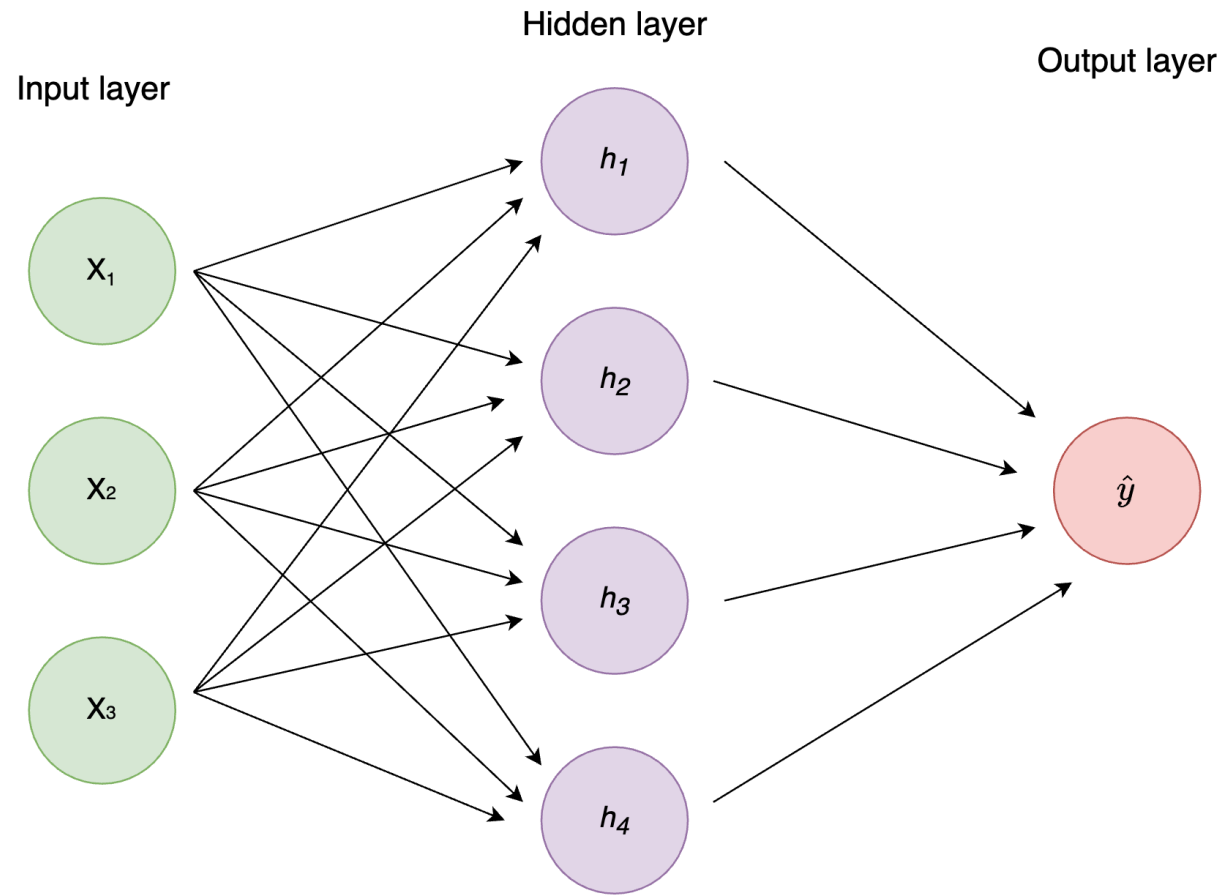
Building a Layer



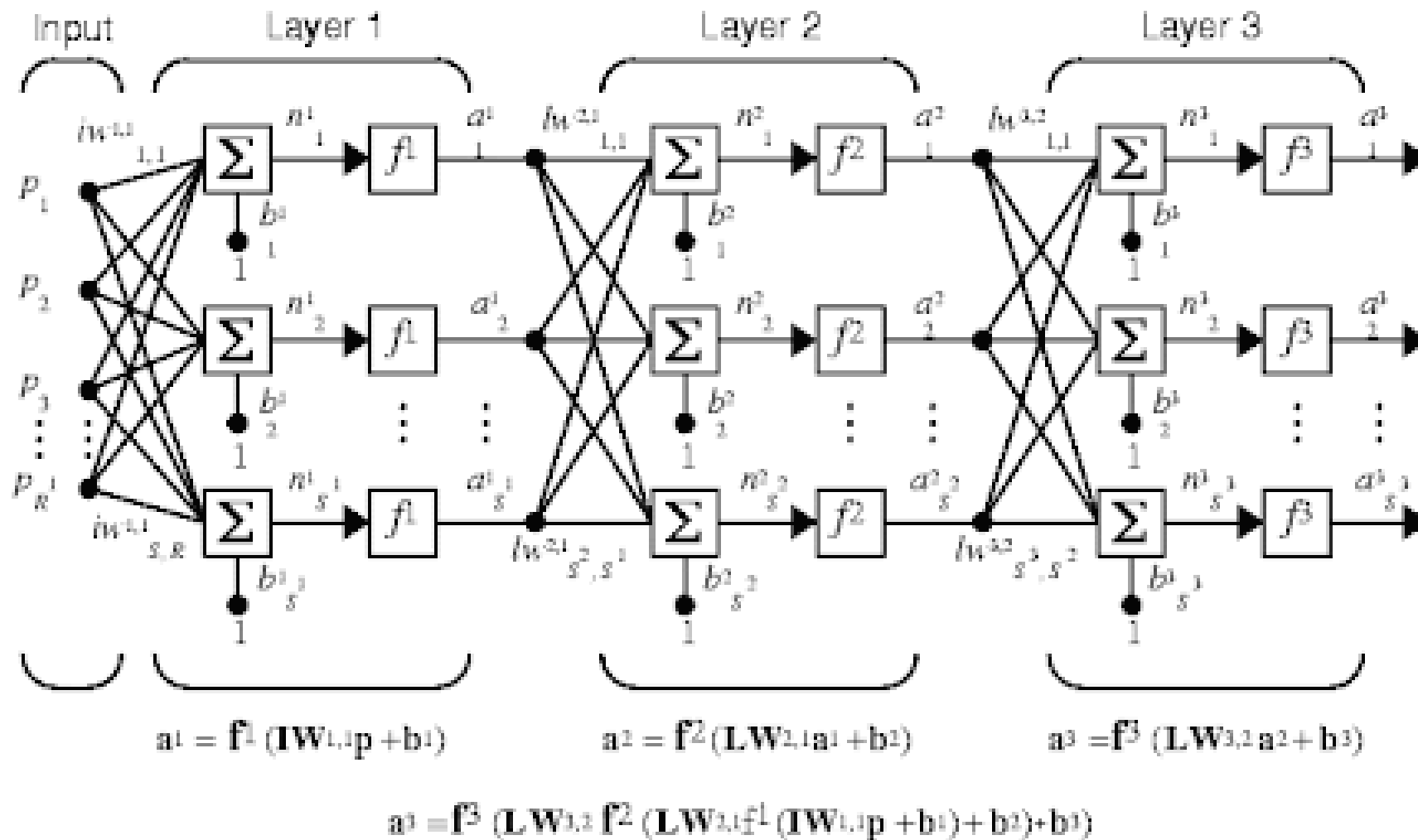
$$[x_1 \ x_2 \ \dots \ x_n] \cdot \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}$$

$$[x_1 \ x_2 \ \dots \ x_n] \cdot \begin{bmatrix} w_{1,1} & \dots & w_{1,n} \\ \vdots & \ddots & \vdots \\ w_{n,1} & \dots & w_{n,n} \end{bmatrix}$$

We Made It!



We Made It!



Open Question

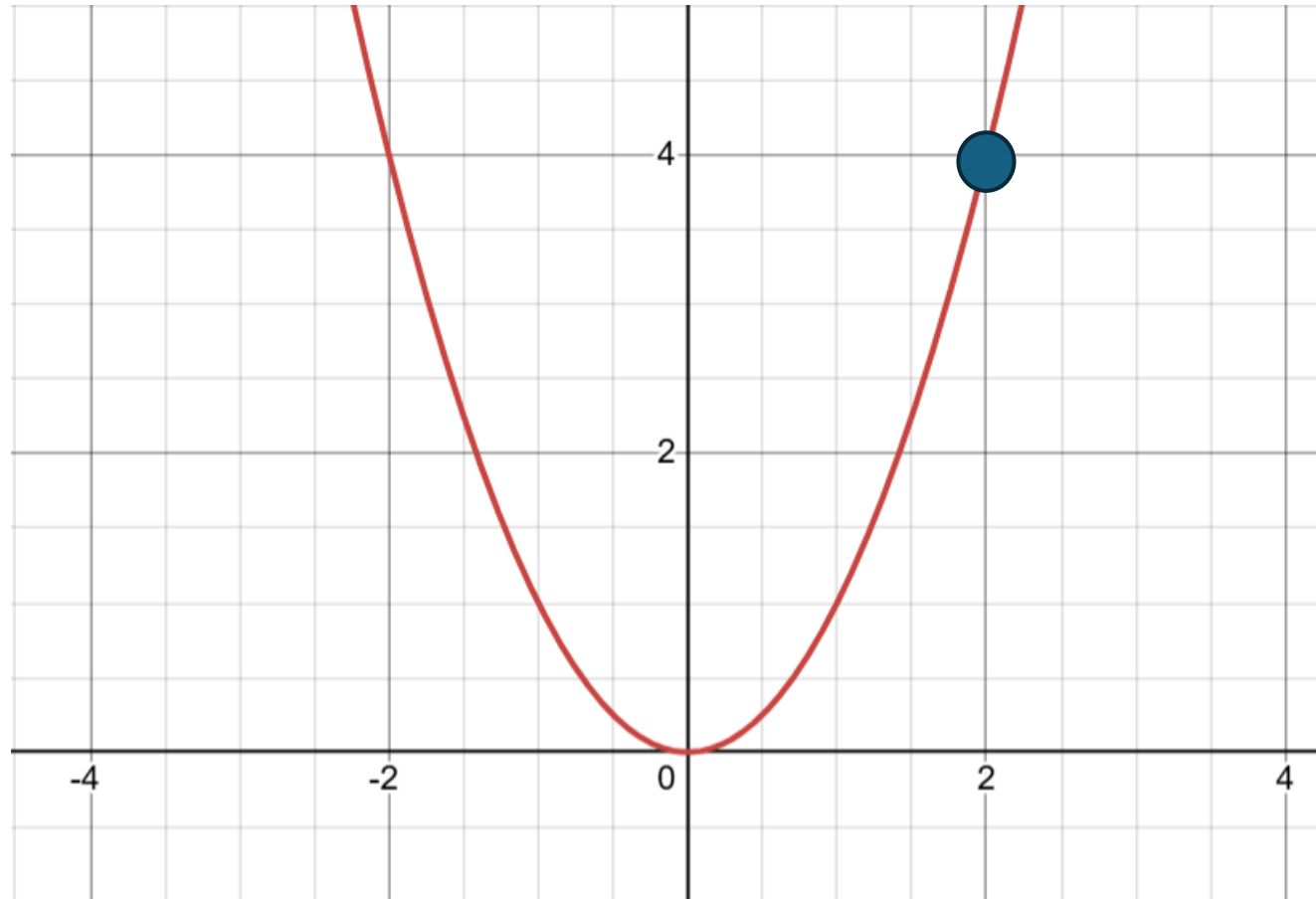
- We just saw how matrices can be used to manipulate data usefully.

But!

- How do we pick which matrices to use?
- How does this connect to backpropagation, our modern AI method?

To do that, we need one last idea to bring it all together.

Derivatives and Gradients



$$y = x^2$$

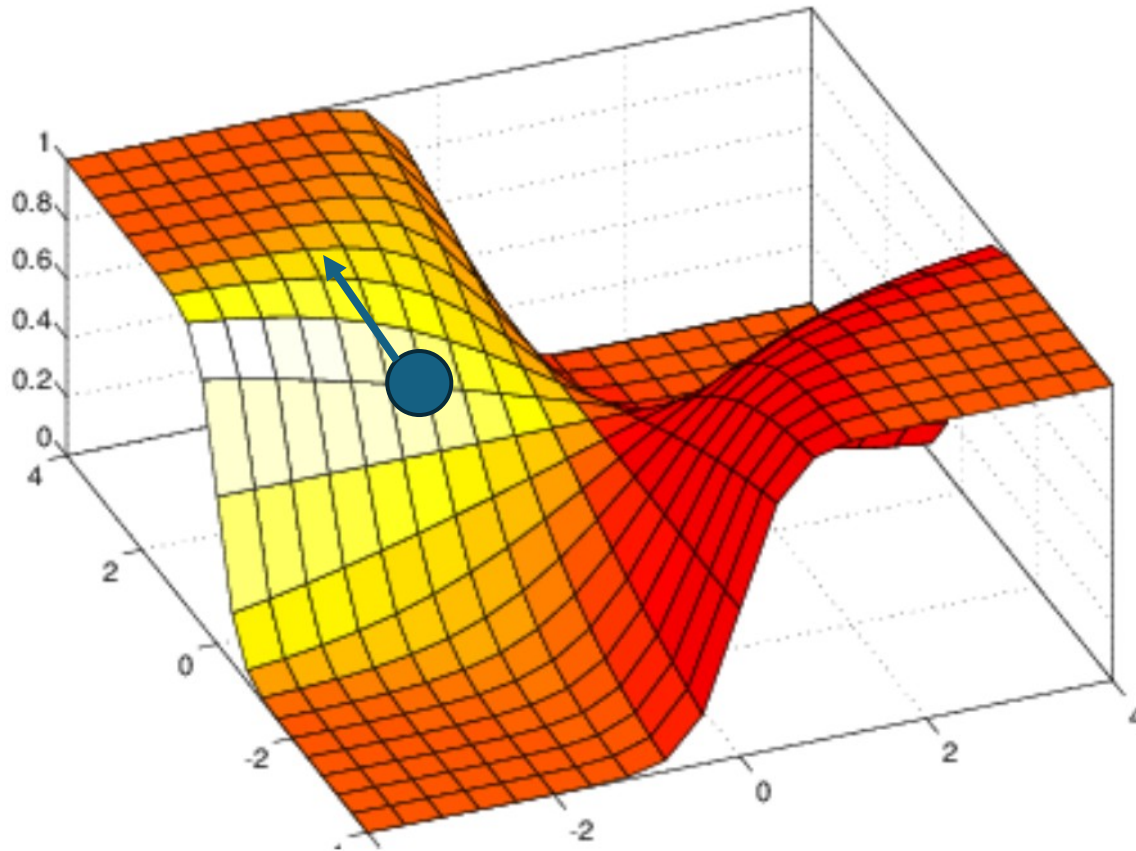
Take the derivative of my function:

$$y = 2x$$

$$x = 2$$

$$\text{slope} = 4$$

Derivatives and Gradients



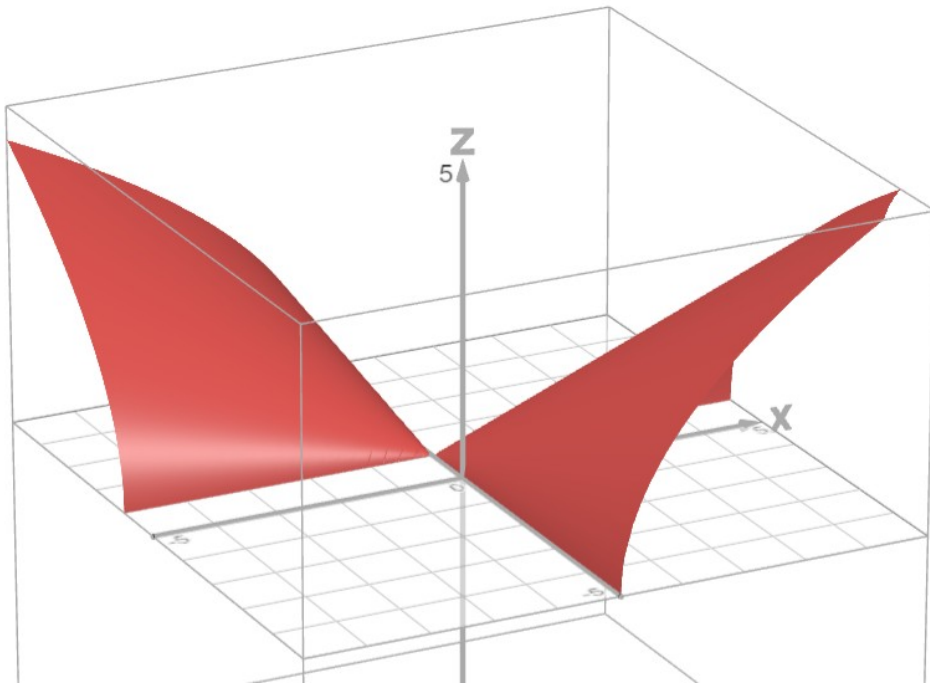
Gradient: direction of greatest change

- Can be on graph of any number of dimensions
- Vector pointing toward this direction

$$\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \vdots \\ \frac{\partial f}{\partial x_n}(p) \end{bmatrix}$$

Gradient example

What is the gradient of $f(x, y, z) = x - xy + z^2$?



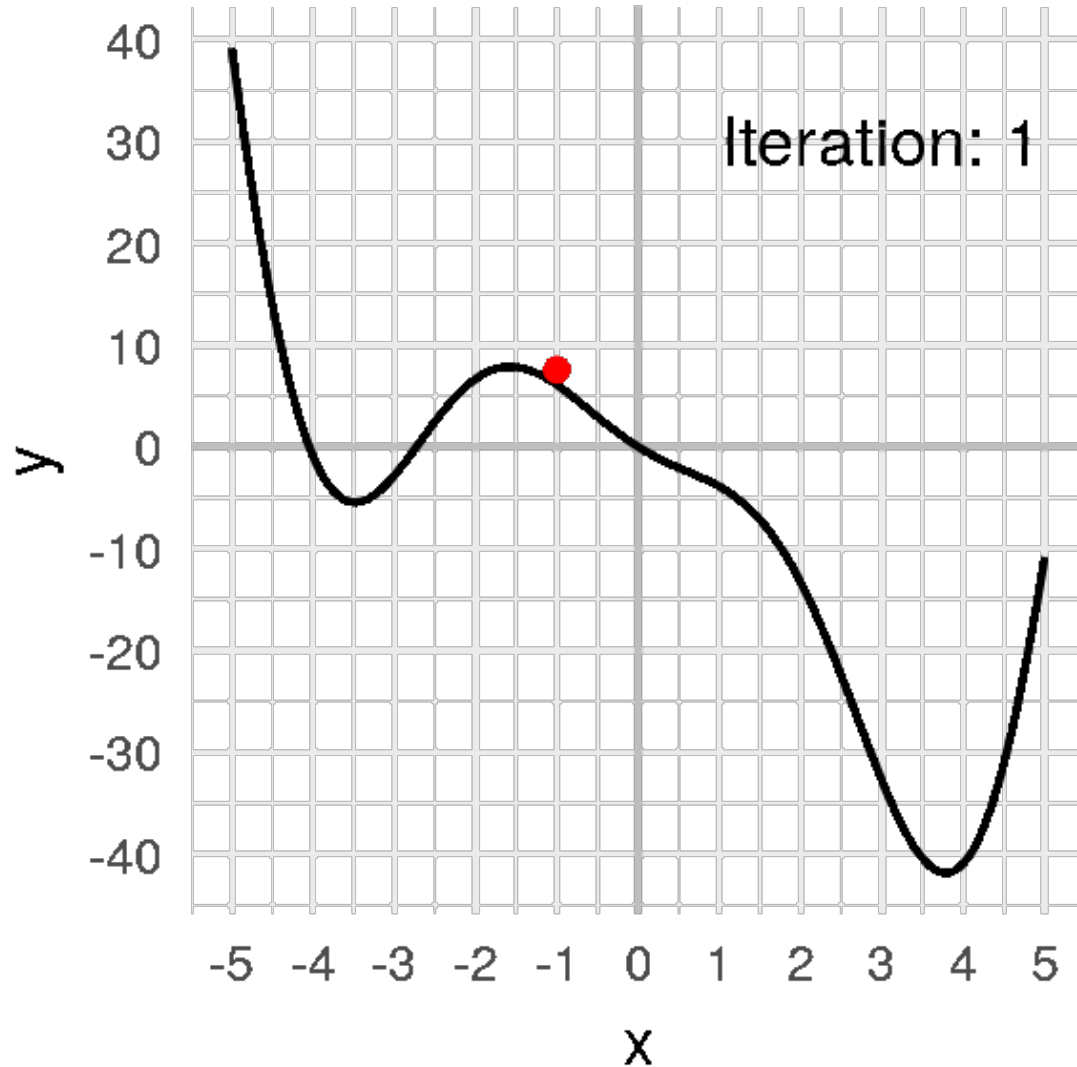
- Holding other variables as a constant, take a derivative
- We get a vector style function telling us what the gradient will be at any point

$$\nabla f(x, y, z) = \begin{bmatrix} 1 - y \\ -x \\ 2z \end{bmatrix}$$

Problems With Computing Gradient

- With so many parameters in these models, near impossible to compute a global gradient function
- Equations are complex and linear, making solving harder too
- Idea: Only compute local gradient, take a step, and repeat

Gradient Descent

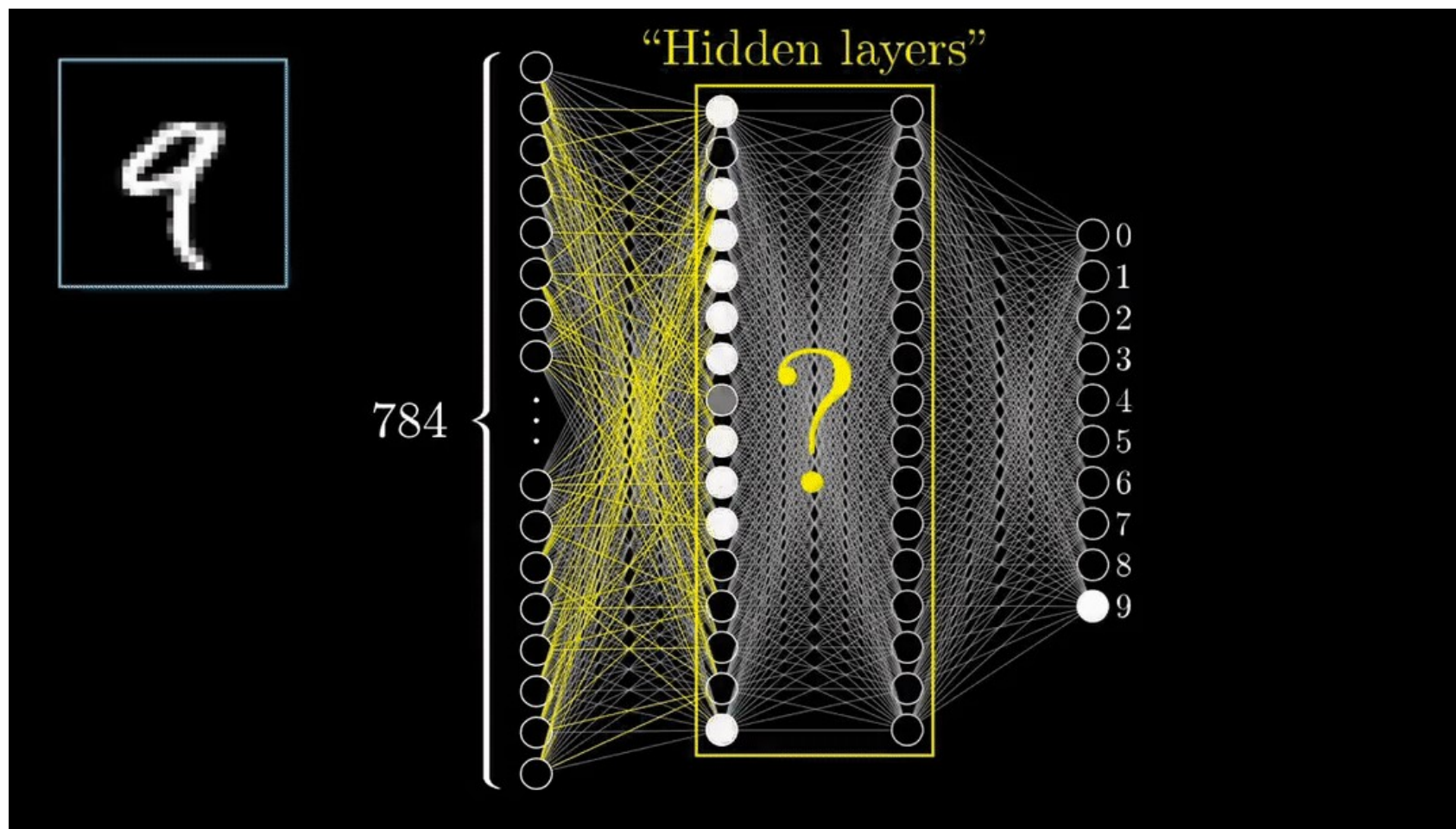


Algorithm:

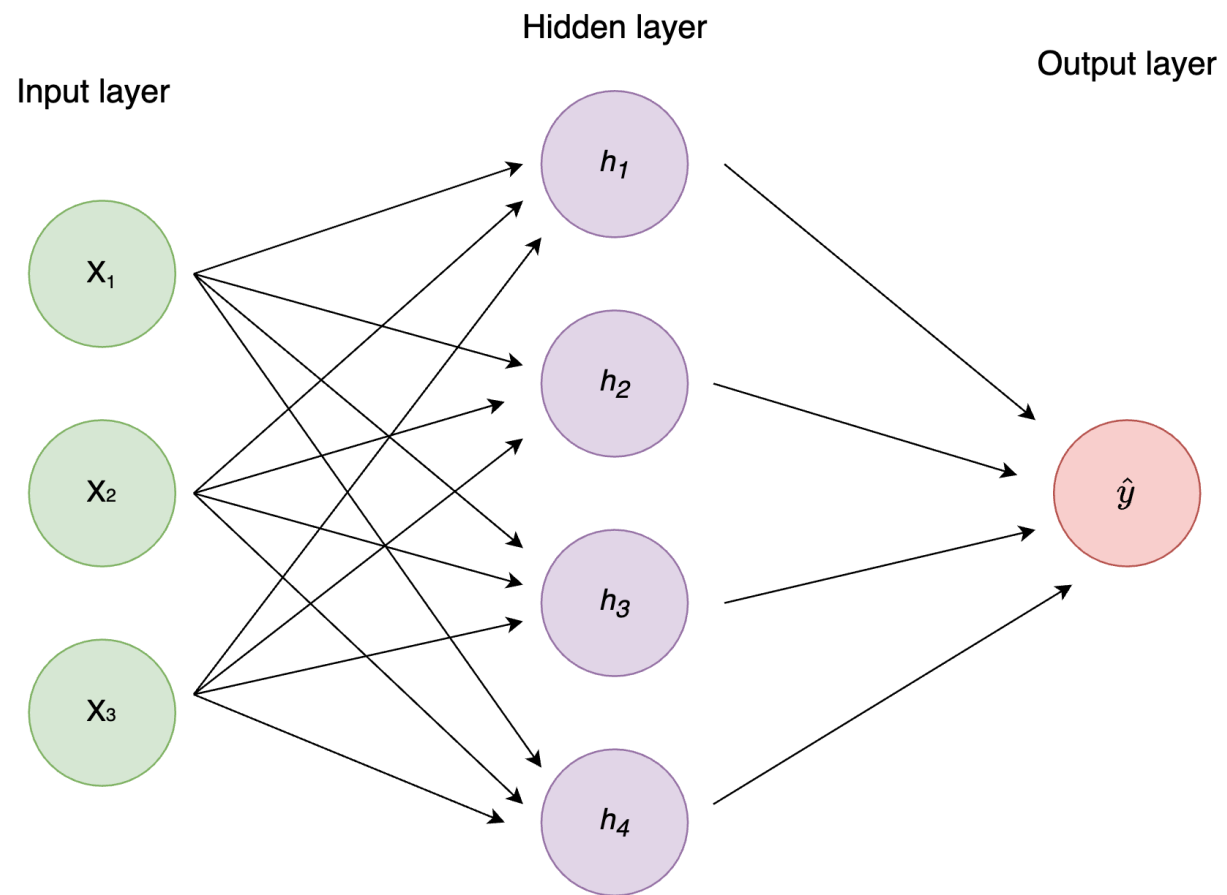
1. Find gradient at start point
2. Move opposite that direction by an amount = gradient magnitude times a constant (learning rate)
3. Repeat until the value between steps changes less than a set amount (or a fixed number of times)

$$\mathbf{a}_{n+1} = \mathbf{a}_n - \eta \nabla f(\mathbf{a}_n)$$

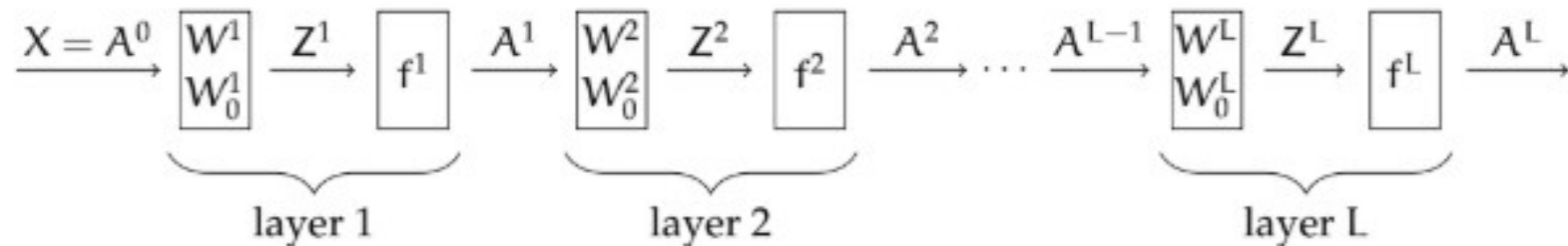
Backpropagation



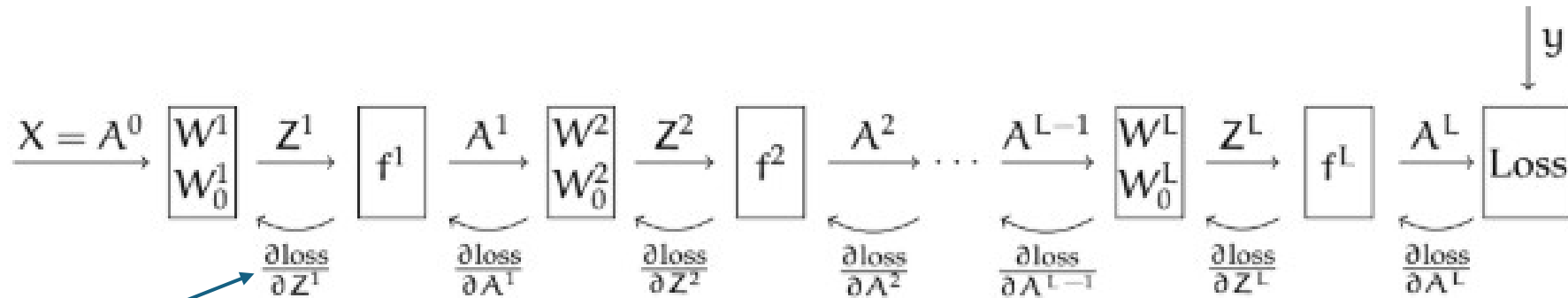
Backpropagation: Updating each layer to descend our loss fn's gradient



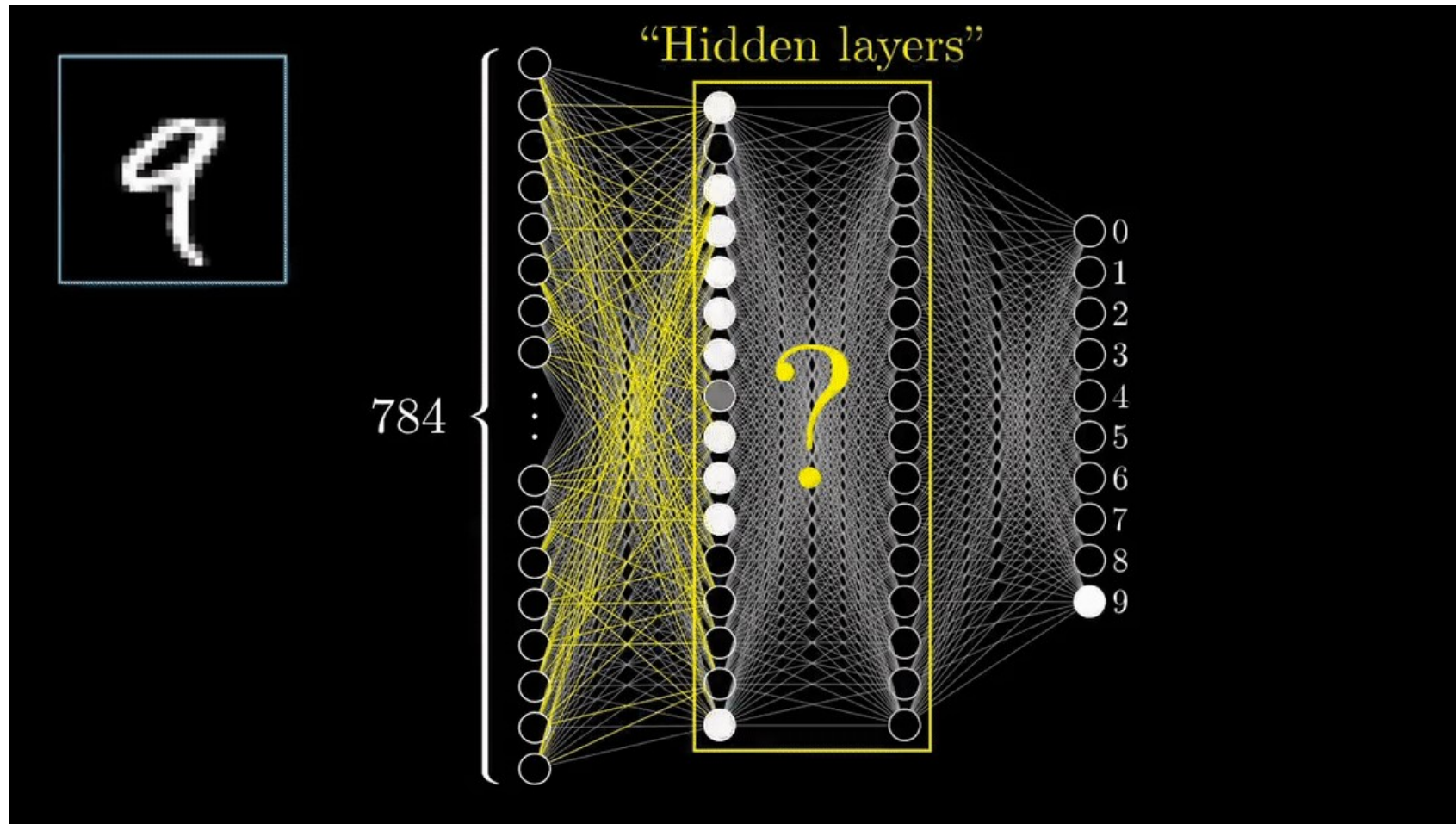
Backpropagation



Find the output of our network by plugging in an example (forward pass)



Find the gradient and update our matrix in each layer.



Now we've got a functioning neural network!

Questions?